

Reflection for C++26

Document #: P2996R2
Date: 2024-02-15
Project: Programming Language C++
Audience: EWG, LEWG
Reply-to: Wyatt Childers
<wcc@edg.com>
Peter Dimov
<pdimov@gmail.com>
Dan Katz
<dkatz85@bloomberg.net>
Barry Revzin
<barry.revzin@gmail.com>
Andrew Sutton
<andrew.n.sutton@gmail.com>
Faisal Vali
<faisalv@gmail.com>
Daveed Vandevoorde
<daveed@edg.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Notable Additions to P1240](#)

[2.2 Why a single opaque reflection type?](#)

[2.3 Implementation Status](#)

[3 Examples](#)

[3.1 Back-And-Forth](#)

[3.2 Selecting Members](#)

[3.3 List of Types to List of Sizes](#)

[3.4 Implementing `make\_integer\_sequence`](#)

[3.5 Getting Class Layout](#)

[3.6 Enum to String](#)

[3.7 Parsing Command-Line Options](#)

[3.8 A Simple Tuple Type](#)

[3.9 A Simple Variant Type](#)

[3.10 Struct to Struct of Arrays](#)

[3.11 Parsing Command-Line Options II](#)

[3.12 A Universal Formatter](#)

[3.13 Implementing `member-wise hash\_append`](#)

[3.14 Converting a Struct to a Tuple](#)

[3.15 Named Tuple](#)

[3.16 Compile-Time Ticket Counter](#)

[3.17 Emulating `typeid` reflection](#)

[4 Proposed Features](#)

[4.1 The Reflection Operator \(\$\Delta\$ \)](#)

[4.1.1 Syntax discussion](#)

[4.2 Splicers \(\[...\]\)](#)

[4.2.1 Range Splicers](#)

[4.2.2 Syntax discussion](#)

[4.3 `std::meta::info`](#)

[4.4 Metafunctions](#)

[4.4.1 Constant evaluation order](#)

[4.4.2 Error-Handling in Reflection](#)

[4.4.3 Handling Aliases](#)

[4.4.4 Freestanding implementations](#)

[4.4.5 Synopsis](#)

[4.4.6 `name\_of`, `display\_name\_of`, `source\_location\_of`](#)

[4.4.7 `type\_of`, `parent\_of`, `dealias`](#)

[4.4.8 `template\_of`, `template\_arguments\_of`](#)

[4.4.9 `members\_of`, `static data members of`, `nonstatic data members of`, `bases of`, `enumerators of`, `subobjects of`](#)

[4.4.10 `substitute`](#)

[4.4.11 `reflect\_invoke`](#)

[4.4.12 `value\_of<T>`](#)

[4.4.13 `test\_type`, `test\_types`](#)

[4.4.14 `reflect\_value`](#)

[4.4.15 `data\_member\_spec`, `define\_class`](#)

[4.4.16 Data Layout Reflection](#)

[4.4.17 Other Type Traits](#)

[5 Proposed Wording](#)

[5.1 Language](#)

[5.1.1 \[lex.phases\] Phases of translation](#)

[5.1.2 \[lex.pptoken\] Preprocessing tokens](#)

[5.1.3 \[lex.operators\] Operators and punctuators](#)

[5.1.4 \[basic.types.general\]](#)

[5.1.5 \[basic.fundamental\] Fundamental types](#)

[5.1.6 \[basic.lookup.argdep\] Argument-dependent name lookup](#)

[5.1.7 \[basic.lookup.qual.general\] General](#)

[5.1.8 \[expr.prim\] Primary expressions](#)

[5.1.9 \[expr.prim.id.qual\] Qualified names](#)

[5.1.10 \[expr.prim.splice\] Expression splicing](#)

[5.1.11 \[expr.unary.general\]](#)

[5.1.12 \[expr.reflect\] The reflection operator](#)

[5.1.13 \[expr.eq\] Equality Operators](#)

[5.1.14 \[expr.const\] Constant Expressions](#)

[5.1.15 \[dcl.typedef\] The `typedef` specifier](#)

[5.1.16 \[dcl.attr.grammar\] Attribute syntax and semantics](#)

[5.1.17 \[temp.names\] Names of template specializations](#)

[5.1.18 \[temp.arg.general\] General](#)

[5.1.19 \[temp.arg.type\] Template type arguments](#)

[5.1.20 \[temp.arg.nontype\] Template non-type arguments](#)

[5.1.21 \[temp.arg.template\] Template template arguments](#)

[5.2 Library](#)

[5.2.1 \[over.built\] Built-in operators](#)
[5.2.2 Header <meta> synopsis](#)
[5.2.3 \[meta.reflection.names\] Reflection names and locations](#)
[5.2.4 \[meta.reflection.queries\] Reflection queries](#)
[5.2.5 \[meta.reflection.member.queries\], Reflection member queries](#)
[5.2.6 \[meta.reflection.unary\] Unary type traits](#)
 [5.2.6.1 \[meta.reflection.unary.cat\] Primary type categories](#)
 [5.2.6.2 \[meta.reflection.unary.comp\] Composite type categories](#)
 [5.2.6.3 \[meta.reflection.unary.prop\] Type properties](#)
 [5.2.6.4 \[meta.reflection.unary.prop.query\] Type property queries](#)
[5.2.7 \[meta.reflection.rel\], Type relations](#)
[5.2.8 \[meta.reflection.trans\], Transformations between types](#)
 [5.2.8.1 \[meta.reflection.trans.cv\], Const-volatile modifications](#)
 [5.2.8.2 \[meta.reflection.trans.ref\], Reference modifications](#)
 [5.2.8.3 \[meta.reflection.trans.sign\], Sign modifications](#)
 [5.2.8.4 \[meta.reflection.trans.arr\], Array modifications](#)
 [5.2.8.5 \[meta.reflection.trans.ptr\], Pointer modifications](#)
 [5.2.8.6 \[meta.reflection.trans.other\], Other transformations](#)

[6 References](#)

§ 1 Revision History

Since [\[P2996R1\]](#), several changes to the overall library API:

- added `qualified_name_of` (to partner with `name_of`)
- removed `is_static` for being ambiguous, added `has_internal_linkage` (and `has_linkage` and `has_external_linkage`) and `is_static_member` instead
- added `is_class_member`, `is_namespace_member`, and `is_concept`
- added `reflect_invoke`
- added [all the type traits](#)

Other paper changes: * some updates to examples, including a new examples which add a [named tuple](#) and [emulate typeful reflection](#). * more discussion of syntax, constant evaluation order, aliases, and freestanding. * adding lots of wording

Since [\[P2996R0\]](#):

- added links to Compiler Explorer demonstrating just about all of the examples
- respecified `synth_struct` to `define_class`
- respecified a few metafunctions to be functions instead of function templates
- introduced section on error handling mechanism and our preference for exceptions (removing invalid reflections)
- added ticket counter and variant examples
- collapsed `entity_ref` and `pointer_to_member` into `value_of`

§ 2 Introduction

This is a proposal for a reduced initial set of features to support static reflection in C++. Specifically, we are mostly proposing a subset of features suggested in [\[P1240R2\]](#):

- the representation of program elements via constant-expressions producing *reflection values* — *reflections* for short — of an opaque type `std::meta::info`,
- a *reflection operator* (prefix `^`) that produces a reflection value for its operand construct,
- a number of `constexpr` *metafunctions* to work with reflections (including deriving other reflections), and
- constructs called *splicers* to produce grammatical elements from reflections (e.g., `[ : refl : ]`).

(Note that this aims at something a little broader than pure “reflection”. We not only want to observe the structure of the program: We also want to ease generating code that depends on those observations. That combination is sometimes referred to as “reflective metaprogramming”, but within WG21 discussion the term “reflection” has often been used informally to refer to the same general idea.)

This proposal is not intended to be the end-game as far as reflection and compile-time metaprogramming are concerned. Instead, we expect it will be a useful core around which more powerful features will be added incrementally over time. In particular, we believe that most or all the remaining features explored in P1240R2 and that code injection (along the lines described in [\[P2237R0\]](#)) are desirable directions to pursue.

Our choice to start with something smaller is primarily motivated by the belief that that improves the chances of these facilities making it into the language sooner rather than later.

§ 2.1 Notable Additions to P1240

While we tried to select a useful subset of the P1240 features, we also made a few additions and changes. Most of those changes are minor. For example, we added a `std::meta::test_type` interface that makes it convenient to use existing standard type predicates (such as `is_class_v`) in reflection computations.

One addition does stand out, however: We have added metafunctions that permit the synthesis of simple struct and union types. While it is not nearly as powerful as generalized code injection (see [\[P2237R0\]](#)), it can be remarkably effective in practice.

§ 2.2 Why a single opaque reflection type?

Perhaps the most common suggestion made regarding the framework outlined in P1240 is to switch from the single `std::meta::info` type to a family of types covering various language elements (e.g., `std::meta::variable`, `std::meta::type`, etc.).

We believe that doing so would be a mistake with very serious consequences for the future of C++.

Specifically, it would codify the language design into the type system. We know from experience that it has been quasi-impossible to change the semantics of standard types once they were standardized, and there is no reason to think that such evolution would become easier in the future. Suppose for example that we had standardized a reflection type `std::meta::variable` in C++03 to represent what the standard called “variables” at the time. In C++11, the term “variable” was extended to include “references”. Such a change would have been difficult to do given that C++ by then likely would have had plenty of code that depended on a type arrangement around the more restricted definition of “variable”. That scenario is clearly backward-looking, but there is no reason to believe that similar changes might not be wanted in the future and we strongly believe that it behooves us to avoid adding undue constraints on the evolution of the language.

Other advantages of a single opaque type include:

- it makes no assumptions about the representation used within the implementation (e.g., it doesn't advantage one compiler over another),
- it is trivially extensible (no types need to be added to represent additional language elements and meta-elements as the language evolves), and
- it allows convenient collections of heterogeneous constructs without having to surface reference semantics (e.g., a `std::vector<std::meta::info>` can easily represent a mixed template argument list — containing types and nontypes — without fear of slicing values).

§ 2.3 Implementation Status

Lock3 implemented the equivalent of much that is proposed here in a fork of Clang (specifically, it worked with the P1240 proposal, but also included several other capabilities including a first-class injection mechanism).

EDG has an ongoing implementation of this proposal that is currently available on Compiler Explorer (thank you, Matt Godbolt). Nearly all of the examples below have links to compiler explorer demonstrating them.

The implementation is not complete (notably, for debugging purposes, `name_of(^int)` yields an empty string and `name_of(^std::optional<std::string>)` yields `"optional"`, neither of which are what we want). The implementation will evolve along with this paper. The EDG implementation also lacks some of the other language features we would like to be able to take advantage of. In particular, it does not support expansion statements. A workaround that will be used in the linked implementations of examples is the following facility:

```
namespace __impl {
    template<auto... vals>
    struct replicator_type {
        template<typename F>
        constexpr void operator>>(F body) const {
            (body.template operator()<vals>(), ...);
        }
    };

    template<auto... vals>
    replicator_type<vals...> replicator = {};
}

template<typename R>
constexpr auto expand(R range) {
    std::vector<std::meta::info> args;
    for (auto r : range) {
        args.push_back(reflect_value(r));
    }
    return substitute(^__impl::replicator, args);
}
```

Used like:

With expansion statements	With expand workaround
<pre>template <typename E> requires std::is_enum_v<E> constexpr std::string enum_to_string(E value) { template for (constexpr auto e : std::meta::enumerators_of(^E)) { if (value == [:e:]) {</pre>	<pre>template<typename E> requires std::is_enum_v<E> constexpr std::string enum_to_string(E value) { std::string result = "<unnamed>"; [:expand(std::meta::enumerators_of(^E)):] >> [&<auto e>{ if (value == [:e:]) { result = std::meta::name_of(e);</pre>

With expansion statements	With expand workaround
<pre> return std::string(std::meta::name_of(e)); } } return "<unnamed>"; } </pre>	<pre> } }; return result; } </pre>

§ 3 Examples

We start with a number of examples that show off what is possible with the proposed set of features. It is expected that these are mostly self-explanatory. Read ahead to the next sections for a more systematic description of each element of this proposal.

A number of our examples here show a few other language features that we hope to progress at the same time. This facility does not strictly rely on these features, and it is possible to do without them - but it would greatly help the usability experience if those could be adopted as well:

- expansion statements [[P1306R1](#)]
- non-transient constexpr allocation [[P0784R7](#)] [[P1974R0](#)] [[P2670R1](#)]

§ 3.1 Back-And-Forth

Our first example is not meant to be compelling but to show how to go back and forth between the reflection domain and the grammatical domain:

```

constexpr auto r = ^int;
typename[:r:] x = 42;      // Same as: int x = 42;
typename[:^char:] c = '*'; // Same as: char c = '*';

```

The `typename` prefix can be omitted in the same contexts as with dependent qualified names (i.e., in what the standard calls *type-only contexts*). For example:

```

using MyType = [:sizeof(int)<sizeof(long)? ^long : ^int:]; // Implicit "typename"
                prefix.

```

[On Compiler Explorer.](#)

§ 3.2 Selecting Members

Our second example enables selecting a member “by number” for a specific type:

```

struct S { unsigned i:2, j:6; };

constexpr auto member_number(int n) {
    if (n == 0) return ^S::i;
    else if (n == 1) return ^S::j;
}

int main() {
    S s{0, 0};
    s[:member_number(1):] = 42; // Same as: s.j = 42;
    s[:member_number(5):] = 0;  // Error (member_number(5) is not a constant).
}

```

This example also illustrates that bit fields are not beyond the reach of this proposal.

[On Compiler Explorer](#)

Note that a “member access splice” like `s.[ :member_number(1): ]` is a more direct member access mechanism than the traditional syntax. It doesn’t involve member name lookup, access checking, or — if the spliced reflection value denotes a member function — overload resolution.

This proposal includes a number of consteval “metafunctions” that enable the introspection of various language constructs. Among those metafunctions is `std::meta::nonstatic_data_members_of` which returns a vector of reflection values that describe the nonstatic members of a given type. We could thus rewrite the above example as:

```
struct S { unsigned i:2, j:6; };

consteval auto member_number(int n) {
    return std::meta::nonstatic_data_members_of(^S)[n];
}

int main() {
    S s{0, 0};
    s.[ :member_number(1): ] = 42; // Same as: s.j = 42;
    s.[ :member_number(5): ] = 0;  // Error (member_number(5) is not a constant).
}
```

[On Compiler Explorer](#)

This proposal specifies that namespace `std::meta` is associated with the reflection type (`std::meta::info`); the `std::meta::` qualification can therefore be omitted in the example above.

Another frequently-useful metafunction is `std::meta::name_of`, which returns a `std::string_view` describing the unqualified name of an entity denoted by a given reflection value. With such a facility, we could conceivably access nonstatic data members “by string”:

```
struct S { unsigned i:2, j:6; };

consteval auto member_named(std::string_view name) {
    for (std::meta::info field : nonstatic_data_members_of(^S)) {
        if (name_of(field) == name) return field;
    }
}

int main() {
    S s{0, 0};
    s.[ :member_named("j"): ] = 42; // Same as: s.j = 42;
    s.[ :member_named("x"): ] = 0;  // Error (member_named("x") is not a constant).
}
```

[On Compiler Explorer](#)

§ 3.3 List of Types to List of Sizes

Here, sizes will be a `std::array<std::size_t, 3>` initialized with `{sizeof(int), sizeof(float), sizeof(double)}`:

```
constexpr std::array types = {^int, ^float, ^double};
constexpr std::array sizes = []{
    std::array<std::size_t, types.size()> r;
    std::ranges::transform(types, r.begin(), std::meta::size_of);
}
```

```
    return r;
}();
```

Compare this to the following type-based approach, which produces the same array sizes:

```
template<class...> struct list {};

using types = list<int, float, double>;

constexpr auto sizes = [<template<class...> class L, class... T>(L<T...>) {
    return std::array<std::size_t, sizeof...(T)>{{ sizeof(T)... }};
}](types{});
```

[On Compiler Explorer.](#)

§ 3.4 Implementing make_integer_sequence

We can provide a better implementation of `make_integer_sequence` than a hand-rolled approach using regular template metaprogramming (although standard libraries today rely on an intrinsic for this):

```
#include <utility>
#include <vector>

template<typename T>
constexpr std::meta::info make_integer_seq_refl(T N) {
    std::vector args{^T};
    for (T k = 0; k < N; ++k) {
        args.push_back(std::meta::reflect_value(k));
    }
    return substitute(^std::integer_sequence, args);
}

template<typename T, T N>
using make_integer_sequence = [:make_integer_seq_refl<T>(N):];
```

[On Compiler Explorer.](#)

Note that the memoization implicit in the template substitution process still applies. So having multiple uses of, e.g., `make_integer_sequence<int, 20>` will only involve one evaluation of `make_integer_seq_refl<int>(20)`.

§ 3.5 Getting Class Layout

```
struct member_descriptor
{
    std::size_t offset;
    std::size_t size;
};

// returns std::array<member_descriptor, N>
template <typename S>
constexpr auto get_layout() {
    constexpr auto members = nonstatic_data_members_of(^S);
    std::array<member_descriptor, members.size()> layout;
    for (int i = 0; i < members.size(); ++i) {
        layout[i] = {.offset=offset_of(members[i]), .size=size_of(members[i])};
    }
    return layout;
}
```

```

struct X
{
    char a;
    int b;
    double c;
};

/*constexpr*/ auto Xd = get_layout<X>();

/*
where Xd would be std::array<member_descriptor, 3>{{
    { 0, 1 }, { 4, 4 }, { 8, 8 }
}}
*/

```

[On Compiler Explorer.](#)

§ 3.6 Enum to String

One of the most commonly requested facilities is to convert an enum value to a string (this example relies on expansion statements):

```

template <typename E>
requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
    template for (constexpr auto e : std::meta::enumerators_of(^E)) {
        if (value == [:e:]) {
            return std::string(std::meta::name_of(e));
        }
    }

    return "<unnamed>";
}

enum Color { red, green, blue };
static_assert(enum_to_string(Color::red) == "red");
static_assert(enum_to_string(Color(42)) == "<unnamed>");

```

We can also do the reverse in pretty much the same way:

```

template <typename E>
requires std::is_enum_v<E>
constexpr std::optional<E> string_to_enum(std::string_view name) {
    template for (constexpr auto e : std::meta::enumerators_of(^E)) {
        if (name == std::meta::name_of(e)) {
            return [:e:];
        }
    }

    return std::nullopt;
}

```

But we don't have to use expansion statements - we can also use algorithms. For instance, `enum_to_string` can also be implemented this way (this example relies on non-transient `constexpr` allocation), which also demonstrates choosing a different algorithm based on the number of enumerators:

```

template <typename E>
requires std::is_enum_v<E>
constexpr std::string enum_to_string(E value) {
    constexpr auto get_pairs = []{

```

```

return std::meta::enumerators_of(^E)
| std::views::transform([](std::meta::info e){
    return std::pair<E, std::string>(std::meta::value_of<E>(e),
    std::meta::name_of(e));
})
};

constexpr auto get_name = [](E value) -> std::optional<std::string> {
    if constexpr (enumerators_of(^E).size() <= 7) {
        // if there aren't many enumerators, use a vector with find_if()
        constexpr auto enumerators = get_pairs() | std::ranges::to<std::vector>();
        auto it = std::ranges::find_if(enumerators, [value](auto const& pr){
            return pr.first == value;
        });
        if (it == enumerators.end()) {
            return std::nullopt;
        } else {
            return it->second;
        }
    } else {
        // if there are lots of enumerators, use a map with find()
        constexpr auto enumerators = get_pairs() | std::ranges::to<std::map>();
        auto it = enumerators.find(value);
        if (it == enumerators.end()) {
            return std::nullopt;
        } else {
            return it->second;
        }
    }
};

return get_name(value).value_or("<unnamed>");
}

```

Note that this last version has lower complexity: While the versions using an expansion statement use an expected $O(N)$ number of comparisons to find the matching entry, a `std::map` achieves the same with $O(\log(N))$ complexity (where N is the number of enumerator constants).

[On Compiler Explorer.](#)

Many many variations of these functions are possible and beneficial depending on the needs of the client code. For example:

- the "<unnamed>" case could instead output a valid cast expression like `"E(5)"`
- a compact two-way persistent data structure could be generated to support both `enum_to_string` and `string_to_enum` with a minimal footprint
- etc.

§ 3.7 Parsing Command-Line Options

Our next example shows how a command-line option parser could work by automatically inferring flags based on member names. A real command-line parser would of course be more complex, this is just the beginning.

```

template<typename Opts>
auto parse_options(std::span<std::string_view const> args) -> Opts {
    Opts opts;
    template for (constexpr auto dm : nonstatic_data_members_of(^Opts)) {
        auto it = std::ranges::find_if(args,

```

```

    [](std::string_view arg){
        return arg.starts_with("--") && arg.substr(2) == name_of(dm);
    });

    if (it == args.end()) {
        // no option provided, use default
        continue;
    } else if (it + 1 == args.end()) {
        std::print(stderr, "Option {} is missing a value\n", *it);
        std::exit(EXIT_FAILURE);
    }

    using T = typename[:type_of(dm):];
    auto iss = std::ispanstream(it[1]);
    if (iss >> opts[:dm:]; !iss) {
        std::print(stderr, "Failed to parse option {} into a {}\n", *it,
            display_name_of(^T));
        std::exit(EXIT_FAILURE);
    }
}
return opts;
}

struct MyOpts {
    std::string file_name = "input.txt"; // Option "--file_name <string>"
    int count = 1; // Option "--count <int>"
};

int main(int argc, char *argv[]) {
    MyOpts opts = parse_options<MyOpts>(std::vector<std::string_view>(argv+1,
        argv+argc));
    // ...
}

```

This example is based on a presentation by Matúš Chochlík.

[On Compiler Explorer.](#)

§ 3.8 A Simple Tuple Type

```

#include <meta>

template<typename... Ts> struct Tuple {
    struct storage;

    static_assert(is_type(define_class(^storage, {data_member_spec(^Ts)...})));
    storage data;

    Tuple(): data{} {}
    Tuple(Ts const& ...vs): data{ vs... } {}
};

template<typename... Ts>
struct std::tuple_size<Tuple<Ts...>>: public integral_constant<size_t, sizeof...
    (Ts)> {};

template<std::size_t I, typename... Ts>
struct std::tuple_element<I, Tuple<Ts...>> {
    static constexpr std::array types = {^Ts...};
    using type = [: types[I] :];
};

```

```

constexpr std::meta::info get_nth_field(std::meta::info r, std::size_t n) {
    return nonstatic_data_members_of(r)[n];
}

template<std::size_t I, typename... Ts>
constexpr auto get(Tuple<Ts...> &t) noexcept -> std::tuple_element_t<I,
    Tuple<Ts...>>& {
    return t.data[:get_nth_field(^decltype(t.data), I):];
}
// Similarly for other value categories...

```

This example uses a “magic” `std::meta::define_class` template along with member reflection through the `nonstatic_data_members_of` metafunction to implement a `std::tuple`-like type without the usual complex and costly template metaprogramming tricks that that involves when these facilities are not available. `define_class` takes a reflection for an incomplete class or union plus a vector of nonstatic data member descriptions, and completes the given class or union type to have the described members.

[On Compiler Explorer.](#)

§ 3.9 A Simple Variant Type

Similarly to how we can implement a tuple using `define_class` to create on the fly a type with one member for each `Ts...`, we can implement a variant that simply defines a `union` instead of a `struct`. One difference here is how the destructor of a `union` is currently defined:

```

union U1 {
    int i;
    char c;
};

union U2 {
    int i;
    std::string s;
};

```

`U1` has a trivial destructor, but `U2`’s destructor is defined as deleted (because `std::string` has a non-trivial destructor). This is a problem because we need to define this thing... somehow. However, for the purposes of `define_class`, there really is only one reasonable option to choose here:

```

template <class... Ts>
union U {
    // all of our members
    Ts... members;

    // a defaulted destructor if all of the types are trivially destructible
    constexpr ~U() requires (std::is_trivially_destructible_v<Ts> && ...) = default;

    // ... otherwise a destructor that does nothing
    constexpr ~U() { }
};

```

If we make `define_class` for a `union` have this behavior, then we can implement a variant in a much more straightforward way than in current implementations. This is not a complete implementation of `std::variant` (and cheats using libstdc++ internals, and also uses Boost.Mp11’s `mp_with_index`) but should demonstrate the idea:

```

template <typename... Ts>
class Variant {

```

```

union Storage;
struct Empty { };

static_assert(is_type(define_class(^Storage, {
    data_member_spec(^Empty, {.name="empty"}),
    data_member_spec(^Ts)...
})));

static constexpr std::array<std::meta::info, sizeof...(Ts)> types = {^Ts...};

static constexpr std::meta::info get_nth_field(std::size_t n) {
    return nonstatic_data_members_of(^Storage)[n+1];
}

Storage storage_;
int index_ = -1;

// cheat: use libstdc++'s implementation
template <typename T>
static constexpr size_t accepted_index =
    std::__detail::__variant::__accepted_index<T, std::variant<Ts...>>;

template <class F>
constexpr auto with_index(F&& f) const -> decltype(auto) {
    return mp_with_index<sizeof...(Ts)>(index_, (F&&)f);
}

public:
    constexpr Variant() requires std::is_default_constructible_v<[: types[0] :]>
        // should this work: storage_{. [: get_nth_field(0) :]}{}
        : storage_{.empty={}}
        , index_(0)
    {
        std::construct_at(&storage_.[: get_nth_field(0) :]);
    }

    constexpr ~Variant() requires (std::is_trivially_destructible_v<Ts> and ...) =
        default;
    constexpr ~Variant() {
        if (index_ != -1) {
            with_index([&](auto I){
                std::destroy_at(&storage_.[: get_nth_field(I) :]);
            });
        }
    }

    template <typename T, size_t I = accepted_index<T&&>>
        requires (!std::is_base_of_v<Variant, std::decay_t<T>>)
    constexpr Variant(T&& t)
        : storage_{.empty={}}
        , index_(-1)
    {
        std::construct_at(&storage_.[: get_nth_field(I) :], (T&&)t);
        index_ = (int)I;
    }

// you can't actually express this constraint nicely until P2963
    constexpr Variant(Variant const&) requires (std::is_trivially_copyable_v<Ts> and
        ...) = default;
    constexpr Variant(Variant const& rhs)
        requires ((std::is_copy_constructible_v<Ts> and ...)
            and not (std::is_trivially_copyable_v<Ts> and ...))

```

```

        : storage_{.empty={}}
        , index_(-1)
    {
        rhs.with_index([&](auto I){
            constexpr auto field = get_nth_field(I);
            std::construct_at(&storage_.[: field :], rhs.storage_.[: field :]);
            index_ = I;
        });
    }

    constexpr auto index() const -> int { return index_; }

    template <class F>
    constexpr auto visit(F&& f) const -> decltype(auto) {
        if (index_ == -1) {
            throw std::bad_variant_access();
        }

        return mp_with_index<sizeof...(Ts)>(index_, [&](auto I) -> decltype(auto) {
            return std::invoke((F&&)f, storage_.[: get_nth_field(I) :]);
        });
    }
};

```

Effectively, `Variant<T, U>` synthesizes a union type `Storage` which looks like this:

```

union Storage {
    Empty empty;
    T unnamed0;
    U unnamed1;

    ~Storage() requires std::is_trivially_destructible_v<T> &&
        std::is_trivially_destructible_v<U> = default;
    ~Storage() { }
}

```

The question here is whether we should be able to directly initialize members of a defined union using a splicer, as in:

```

: storage{.[: get_nth_field(0) :]={}}

```

Arguably, the answer should be yes - this would be consistent with how other accesses work.

[On Compiler Explorer.](#)

§ 3.10 Struct to Struct of Arrays

```

#include <meta>
#include <array>

template <typename T, std::size_t N>
struct struct_of_arrays_impl;

constexpr auto make_struct_of_arrays(std::meta::info type,
                                     std::meta::info N) -> std::meta::info {
    std::vector<std::meta::info> old_members = nonstatic_data_members_of(type);
    std::vector<std::meta::info> new_members = {};
    for (std::meta::info member : old_members) {
        auto array_type = substitute(^std::array, {type_of(member), N });
        auto mem_descr = data_member_spec(array_type, {.name = name_of(member)});
    }
}

```

```

        new_members.push_back(mem_descr);
    }
    return std::meta::define_class(
        substitute(^struct_of_arrays_impl, {type, N}),
        new_members);
}

template <typename T, size_t N>
using struct_of_arrays = [: make_struct_of_arrays(^T, ^N) :];

```

Example:

```

struct point {
    float x;
    float y;
    float z;
};

using points = struct_of_arrays<point, 30>;
// equivalent to:
// struct points {
//     std::array<float, 30> x;
//     std::array<float, 30> y;
//     std::array<float, 30> z;
// };

```

Again, the combination of `nonstatic_data_members_of` and `define_class` is put to good use.

[On Compiler Explorer.](#)

§ 3.11 Parsing Command-Line Options II

Now that we've seen a couple examples of using `std::meta::define_class` to create a type, we can create a more sophisticated command-line parser example.

This is the opening example for [clap](#) (Rust's **C**ommand **L**ine **A**rgument **P**arser):

```

struct Args : Clap {
    Option<std::string, {.use_short=true, .use_long=true}> name;
    Option<int, {.use_short=true, .use_long=true}> count = 1;
};

int main(int argc, char** argv) {
    auto opts = Args{}.parse(argc, argv);

    for (int i = 0; i < opts.count; ++i) { // opts.count has type int
        std::print("Hello {}!", opts.name); // opts.name has type std::string
    }
}

```

Which we can implement like this:

```

struct Flags {
    bool use_short;
    bool use_long;
};

template <typename T, Flags flags>
struct Option {
    std::optional<T> initializer = {};
}

```

```

    // some suitable constructors and accessors for flags
};

// convert a type (all of whose non-static data members are specializations of Option)
// to a type that is just the appropriate members.
// For example, if type is a reflection of the Args presented above, then this
// function would evaluate to a reflection of the type
// struct {
//     std::string name;
//     int count;
// }
constexpr auto spec_to_opts(std::meta::info opts,
                           std::meta::info spec) -> std::meta::info {
    std::vector<std::meta::info> new_members;
    for (std::meta::info member : nonstatic_data_members_of(spec)) {
        auto new_type = template_arguments_of(type_of(member))[0];
        new_members.push_back(data_member_spec(new_type, {.name=name_of(member)}));
    }
    return define_class(opts, new_members);
}

struct Clap {
    template <typename Spec>
    auto parse(this Spec const& spec, int argc, char** argv) {
        std::vector<std::string_view> cmdline(argv+1, argv+argc)

        // check if cmdline contains --help, etc.

        struct Opts;
        static_assert(is_type(spec_to_opts(^Opts, ^Spec)));
        Opts opts;

        template for (constexpr auto [sm, om] :
            std::views::zip(nonstatic_data_members_of(^Spec),
                nonstatic_data_members_of(^Opts))) {
            auto const& cur = spec.[:sm:];
            constexpr auto type = type_of(om);

            // find the argument associated with this option
            auto it = std::ranges::find_if(cmdline,
                [&](std::string_view arg){
                    return (cur.use_short && arg.size() == 2 && arg[0] == '-' && arg[1] ==
                        name_of(sm)[0])
                        || (cur.use_long && arg.starts_with("--") && arg.substr(2) ==
                            name_of(sm));
                });

            // no such argument
            if (it == cmdline.end()) {
                if constexpr (has_template_arguments(type) and template_of(type) ==
                    ^std::optional) {
                    // the type is optional, so the argument is too
                    continue;
                } else if (cur.initializer) {
                    // the type isn't optional, but an initializer is provided, use that
                    opts.[:om:] = *cur.initializer;
                    continue;
                } else {
                    std::print(stderr, "Missing required option {}\n", name_of(sm));
                    std::exit(EXIT_FAILURE);
                }
            }
        }
    }
};

```

```

    }
} else if (it + 1 == cmdline.end()) {
    std::print(stderr, "Option {} for {} is missing a value\n", *it, name_of(sm));
    std::exit(EXIT_FAILURE);
}

// found our argument, try to parse it
auto iss = ispanstream(it[1]);
if (iss >> opts.[:om:]; !iss) {
    std::print(stderr, "Failed to parse {:?} into option {} of type {}\n",
        it[1], name_of(sm), display_name_of(type));
    std::exit(EXIT_FAILURE);
}
}
return opts;
}
};

```

[On Compiler Explorer.](#)

§ 3.12 A Universal Formatter

This example is taken from Boost.Describe:

```

struct universal_formatter {
    constexpr auto parse(auto& ctx) { return ctx.begin(); }

    template <typename T>
    auto format(T const& t, auto& ctx) const {
        auto out = std::format_to(ctx.out(), "{}{{{", name_of(^T));

        auto delim = [first=true]() mutable {
            if (!first) {
                *out++ = ',';
                *out++ = ' ';
            }
            first = false;
        };

        template for (constexpr auto base : bases_of(^T)) {
            delim();
            out = std::format_to(out, "{}", static_cast<[:base:] const&>(t));
        }

        template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
            delim();
            out = std::format_to(out, ".{}={}", name_of(mem), t.[:mem:]);
        }

        *out++ = '}';
        return out;
    }
};

struct X { int m1 = 1; };
struct Y { int m2 = 2; };
class Z : public X, private Y { int m3 = 3; int m4 = 4; };

template <> struct std::formatter<X> : universal_formatter { };
template <> struct std::formatter<Y> : universal_formatter { };
template <> struct std::formatter<Z> : universal_formatter { };

```

```
int main() {
    std::println("{} ", Z()); // Z{X{.m1 = 1}, Y{.m2 = 2}, .m3 = 3, .m4 = 4}
}
```

This example is not implemented on compiler explorer at this time, but only because of issues compiling both `std::format` and `fmt::format`.

§ 3.13 Implementing member-wise hash_append

Based on the [\[N3980\]](#) API:

```
template <typename H, typename T> requires std::is_standard_layout_v<T>
void hash_append(H& algo, T const& t) {
    template for (constexpr auto mem : nonstatic_data_members_of(^T)) {
        hash_append(algo, t.[:mem:]);
    }
}
```

§ 3.14 Converting a Struct to a Tuple

This approach requires allowing packs in structured bindings [\[P1061R5\]](#), but can also be written using `std::make_index_sequence`:

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
    constexpr auto members = nonstatic_data_members_of(^T);

    constexpr auto indices = []{
        std::array<int, members.size()> indices;
        std::ranges::iota(indices, 0);
        return indices;
    }();

    constexpr auto [...Is] = indices;
    return std::make_tuple(t.[: members[Is] :]....);
}
```

An alternative approach is:

```
constexpr auto struct_to_tuple_type(info type) -> info {
    return substitute(^std::tuple,
        nonstatic_data_members_of(type)
        | std::ranges::transform(std::meta::type_of)
        | std::ranges::transform(std::meta::remove_cvref)
        | std::ranges::to<std::vector>());
}

template <typename To, typename From, std::meta::info ... members>
constexpr auto struct_to_tuple_helper(From const& from) -> To {
    return To(from.[:members:]....);
}

template<typename From>
constexpr auto get_struct_to_tuple_helper() {
    using To = [: struct_to_tuple_type(^From): ];

    std::vector args = {^To, ^From};
    for (auto mem : nonstatic_data_members_of(^From)) {
```

```

    args.push_back(reflect_value(mem));
}

/*
Alternatively, with Ranges:
args.append_range(
    nonstatic_data_members_of(^From)
    | std::views::transform(std::meta::reflect_value)
);
*/

return value_of<To(*)>(From const&)>(
    substitute(^struct_to_tuple_helper, args));
}

template <typename From>
constexpr auto struct_to_tuple(From const& from) {
    return get_struct_to_tuple_helper<From>()(from);
}

```

Here, `struct_to_tuple_type` takes a reflection of a type like `struct { T t; U const& u; V v; }` and returns a reflection of the type `std::tuple<T, U, V>`. That gives us the return type. Then, `struct_to_tuple_helper` is a function template that does the actual conversion — which it can do by having all the reflections of the members as a non-type template parameter pack. This is a `constexpr` function and not a `constexpr` function because in the general case the conversion is a run-time operation. However, determining the instance of `struct_to_tuple_helper` that is needed is a compile-time operation and has to be performed with a `constexpr` function (because the function invokes `nonstatic_data_members_of`), hence the separate function template `get_struct_to_tuple_helper()`.

Everything is put together by using `substitute` to create the instantiation of `struct_to_tuple_helper` that we need, and a compile-time reference to that instance is obtained with `value_of`. Thus `f` is a function reference to the correct specialization of `struct_to_tuple_helper`, which we can simply invoke.

[On Compiler Explorer](#), with a different implementation than either of the above.

§ 3.15 Named Tuple

The tricky thing with implementing a named tuple is actually strings as non-type template parameters. Because you cannot just pass `"x"` into a non-type template parameter of the form `auto v`, that leaves us with two ways of specifying the constituents:

1. Can introduce a pair type so that we can write `make_named_tuple<pair<int, "x">, pair<double, "y">>()`, or
2. Can just do reflections all the way down so that we can write `make_named_tuple<^int, ^"x", ^double, ^"y">()`.

We do not currently support splicing string literals (although that may change in the next revision), and the `pair` approach follows the similar pattern already shown with `define_class` (given a suitable `fixed_string` type):

```

template <class T, fixed_string Name>
struct pair {
    static constexpr auto name() -> std::string_view { return Name.view(); }
    using type = T;
};

template <class... Tags>
constexpr auto make_named_tuple(std::meta::info type, Tags... tags) {

```

```

std::vector<std::meta::info> nsdms;
auto f = [&<class Tag>(Tag tag){
    nsdms.push_back(data_member_spec(
        dealias(^typename Tag::type),
        {.name=Tag::name()}));
};
(f(tags), ...);
return define_class(type, nsdms);
}

struct R;
static_assert(is_type(make_named_tuple(^R, pair<int, "x">{}, pair<double, "y">{})));

static_assert(type_of(nonstatic_data_members_of(^R)[0]) == ^int);
static_assert(type_of(nonstatic_data_members_of(^R)[1]) == ^double);

int main() {
    [[maybe_unused]] auto r = R{.x=1, .y=2.0};
}

```

[On Compiler Explorer.](#)

§ 3.16 Compile-Time Ticket Counter

The features proposed here make it a little easier to update a ticket counter at compile time. This is not an ideal implementation (we'd prefer direct support for compile-time — i.e., `constexpr` — variables), but it shows how compile-time mutable state surfaces in new ways.

```

class TU_Ticket {
    template<int N> struct Helper {
        static constexpr int value = N;
    };
public:
    static constexpr int next() {
        // Search for the next incomplete Helper<k>.
        std::meta::info r;
        for (int k = 0; ++k) {
            r = substitute(^Helper, { std::meta::reflect_value(k) });
            if (is_incomplete_type(r)) break;
        }
        // Return the value of its member. Calling static_data_members_of
        // triggers the instantiation (i.e., completion) of Helper<k>.
        return value_of<int>(static_data_members_of(r)[0]);
    }
};

int x = TU_Ticket::next(); // x initialized to 0.
int y = TU_Ticket::next(); // y initialized to 1.
int z = TU_Ticket::next(); // z initialized to 2.

```

Note that this relies on the fact that a call to `substitute` returns a specialization of a template, but doesn't trigger the instantiation of that specialization. Thus, the only instantiations of `TU_Ticket::Helper` occur because of the call to `nonstatic_data_members_of` (which is a singleton representing the lone value member).

[On Compiler Explorer.](#)

§ 3.17 Emulating typeful reflection

Although we believe a single opaque `std::meta::info` type to be the best and most scalable foundation for reflection, we acknowledge the desire expressed by SG7 for future support for “typeful reflection”. The following demonstrates one possible means of assembling a typeful reflection library, in which different classes of reflections are represented by distinct types, on top of the facilities proposed here.

```
// Represents a 'std::meta::info' constrained by a predicate.
template <std::meta::info Pred>
    requires (type_of(^[:Pred:](^int))) == ^bool)
struct metatype {
    std::meta::info value;

    // Construction is ill-formed unless predicate is satisfied.
    consteval metatype(std::meta::info r) : value(r) {
        if (![:Pred:](r))
            throw "Reflection is not a member of this metatype";
    }

    // Cast to 'std::meta::info' allows values of this type to be spliced.
    consteval operator std::meta::info() const { return value; }

    static consteval bool check(std::meta::info r) { return [:Pred:](r); }
};

// Type representing a "failure to match" any known metatypes.
struct unmatched {
    consteval unmatched(std::meta::info) {}
    static consteval bool check(std::meta::info) { return true; }
};

// Returns the given reflection "enriched" with a more descriptive type.
template <typename... Choices>
consteval std::meta::info enrich(std::meta::info r) {
    // Because we control the type, we know that the constructor taking info is
    // the first constructor. The copy/move constructors are added at the }, so
    // will be the last ones in the list.
    std::array ctors = {members_of(^Choices, std::meta::is_constructor)[0]...,
                       members_of(^unmatched, std::meta::is_constructor)[0]};
    std::array checks = {^Choices::check..., ^unmatched::check};

    std::meta::info choice;
    for (auto [check, ctor] : std::views::zip(checks, ctors))
        if (value_of<bool>(reflect_invoke(check, {reflect_value(r)})))
            return reflect_invoke(ctor, {reflect_value(r)});

    std::unreachable();
}
```

We can leverage this machinery to select different function overloads based on the “type” of reflection provided as an argument.

```
using type_t = metatype<^std::meta::is_type>;
using fn_t = metatype<^std::meta::is_function>;

// Example of a function overloaded for different "types" of reflections.
void PrintKind(type_t) { std::println("type"); }
void PrintKind(fn_t) { std::println("function"); }
void PrintKind(unmatched) { std::println("unknown kind"); }

int main() {
    // Classifies any reflection as one of: Type, Function, or Unmatched.
}
```

```

auto enrich = [](std::meta::info r) { return ::enrich<type_t, fn_t>(r); };

// Demonstration of using 'enrich' to select an overload.
PrintKind([:enrich(^main):]); // "function"
PrintKind([:enrich(^int):]);  // "type"
PrintKind([:enrich(^3):]);    // "unknown kind"
}

```

Note that the metatype class can be generalized to wrap values of any literal type, or to wrap multiple values of possibly different types. This has been used, for instance, to select compile-time overloads based on: whether two integers share the same parity, the presence or absence of a value in an optional, the type of the value held by a variant or an any, or the syntactic form of a compile-time string.

Achieving the same in C++23, with the same generality, would require spelling the argument(s) twice: first to obtain a “classification tag” to use as a template argument, and again to call the function, i.e.,

```

Printer::PrintKind<classify(^int)>(^int).
// or worse...
fn<classify(Arg1, Arg2, Arg3)>(Arg1, Arg2, Arg3).

```

§ 4 Proposed Features

§ 4.1 The Reflection Operator (^)

The reflection operator produces a reflection value from a grammatical construct (its operand):

unary-expression:

```

...
^ ::
^ namespace-name
^ type-id
^ cast-expression

```

Note that *cast-expression* includes *id-expression*, which in turn can designate templates, member names, etc.

The current proposal requires that the *cast-expression* be:

- a *primary-expression* referring to a function or member function, or
- a *primary-expression* referring to a variable, static data member, or structured binding, or
- a *primary-expression* referring to a nonstatic data member, or
- a *primary-expression* referring to a template, or
- a constant-expression.

In a SFINAE context, a failure to substitute the operand of a reflection operator construct causes that construct to not evaluate to constant.

§ 4.1.1 Syntax discussion

The original TS landed on `reflexpr(...)` as the syntax to reflect source constructs and [P1240R0] adopted that syntax as well. As more examples were discussed, it became clear that that syntax was both (a) too “heavy” and (b) insufficiently distinct from a function call. SG7 eventually agreed upon the prefix `^` operator. The “upward arrow”

interpretation of the caret matches the “lift” or “raise” verbs that are sometimes used to describe the reflection operation in other contexts.

The caret already has a meaning as a binary operator in C++ (“exclusive OR”), but that is clearly not conflicting with a prefix operator. In C++/CLI (a Microsoft C++ dialect) the caret is also used as a new kind of *ptr-operator* (9.3.1 [\[dcl.decl.general\]](#)) to declare “[handles](#)”. That is also not conflicting with the use of the caret as a unary operator because C++/CLI uses the usual prefix `*` operator to dereference handled.

Apple also uses the caret in the [syntax “blocks”](#) and unfortunately we believe that does conflict with our proposed use of the caret.

Since the syntax discussions in SG7 landed on the use of the caret, new basic source characters have become available: `@`, ```, and `$`. Of those, `@` seems the most likely substitute for the caret, because `$` is used for splice-like operations in other languages and ``` is suggestive of some kind of quoting (which may be useful in future metaprogramming syntax developments).

Another option might be the use of the backslash (`\`). It currently has a meaning at the end of a line of source code, but we could still use it as a prefix operator with the constraint that the reflected operand has to start on the same source line. If we were to opt for that choice, it could make sense to use the slash (`/`) as a unary operator denoting splicing (see [Splicers](#) below) so that `\` would correspond to “raise” and `/` would correspond to “lower”.

§ 4.2 Splicers (`[ : ... : ]`)

A reflection can be “spliced” into source code using one of several *splicer* forms:

- `[ : r : ]` produces an *expression* evaluating to the entity or constant value represented by `r` in grammatical contexts that permit expressions. In type-only contexts (13.8.1 [\[temp.res.general\]](#)/4), `[ : r : ]` produces a type (and `r` must be the reflection of a type). In contexts that only permit a namespace name, `[ : r : ]` produces a namespace (and `r` must be the reflection of a namespace or alias thereof).
- `typename[ : r : ]` produces a *simple-type-specifier* corresponding to the type represented by `r`.
- `template[ : r : ]` produces a *template-name* corresponding to the template represented by `r`.
- `[ : r : ] ::` produces a *nested-name-specifier* corresponding to the namespace, enumeration type, or class type represented by `r`.

The operand of a splicer is implicitly converted to a `std::meta::info` prvalue (i.e., if the operand expression has a class type that with a conversion function to convert to `std::meta::info`, splicing can still work).

Attempting to splice a reflection value that does not meet the requirement of the splice is ill-formed. For example:

```
typename[ : ^ : ] x = 0; // Error.
```

§ 4.2.1 Range Splicers

The splicers described above all take a single object of type `std::meta::info` (described in more detail below). However, there are many cases where we don’t have a single reflection, we have a range of reflections - and we want to splice them all in one go. For that, we need a different form of splicer: a range splicer.

Construct the [struct-to-tuple](#) example from above. It was demonstrated using a single splice, but it would be simpler if we had a range splice:

With Single Splice	With Range Splice
<pre>template <typename T> constexpr auto struct_to_tuple(T const& t) { constexpr auto members = nonstatic_data_members_of(^T); constexpr auto indices = []{ std::array<int, members.size()> indices; std::ranges::iota(indices, 0); return indices; }(); constexpr auto [...Is] = indices; return std::make_tuple(t.[: members[Is] :]...); }</pre>	<pre>template <typename T> constexpr auto struct_to_tuple(T const& t) { constexpr auto members = nonstatic_data_members_of(^T); return std::make_tuple(t.[: ...members :]...); }</pre>

A range splice, `[ : ... r : ]`, would accept as its argument a constant range of `meta::info`, `r`, and would behave as an unexpanded pack of splices. So the above expression

```
make_tuple(t.[ : ... members : ]...)
```

would evaluate as

```
make_tuple(t.[ :members[0]: ], t.[ :members[1]: ], ..., t.[ :members[N-1]: ])
```

This is a very useful facility indeed!

However, range splicing of dependent arguments is at least an order of magnitude harder to implement than ordinary splicing. We think that not including range splicing gives us a better chance of having reflection in C++26. Especially since, as this paper’s examples demonstrate, a lot can be done without them.

Another way to work around a lack of range splicing would be to implement `with_size<N>(f)`, which would behave like `f(integral_constant<size_t, 0>{}, integral_constant<size_t, 0>{}, ..., integral_constant<size_t, N-1>{})`. Which is enough for a tolerable implementation:

```
template <typename T>
constexpr auto struct_to_tuple(T const& t) {
    constexpr auto members = nonstatic_data_members_of(^T);
    return with_size<members.size()>([&](auto... Is){
        return std::make_tuple(t.[ : members[Is] : ]...);
    });
}
```

(P1240 did propose range splicers.)

§ 4.2.2 Syntax discussion

Early discussions of splice-like constructs (related to the TS design) considered using `unreflexpr(...)` for that purpose. [P1240R0] adopted that option for *expression* splicing, observing that a single splicing syntax could not viably be parsed (some disambiguation is needed to distinguish types and templates). S-7 eventually agreed with the `[ : ... : ]` syntax — with disambiguating tokens such as `typename` where needed — which is a little lighter and more distinctive.

We propose `[ :` and `: ]` be single tokens rather than combinations of `[ , ]`, and `:`. Among others, it simplifies the handling of expressions like `arr[[ : refl() : ]]`. On the flip side, it requires a special rule like the one that was made

to handle `<::` to leave the meaning of `arr[::N]` unchanged and another one to avoid breaking a (somewhat useless) attribute specifier of the form `[[using ns:]]`.

A syntax that is delimited on the left and right is useful here because spliced expressions may involve lower-precedence operators. However, there are other possibilities. For example, now that `$` is available in the basic source character set, we might consider `$<expr>`. This is somewhat natural to those of us that have used systems where `$` is used to expand placeholders in document templates. For example:

```
$select_type(3) *ptr = nullptr;
```

The prefixes `typename` and `template` are only strictly needed in some cases where the operand of the splice is a dependent expression. In our proposal, however, we only make `typename` optional in the same contexts where it would be optional for qualified names with dependent name qualifiers. That has the advantage to catch unfortunate errors while keeping a single rule and helping human readers parse the intended meaning of otherwise ambiguous constructs.

§ 4.3 `std::meta::info`

The type `std::meta::info` can be defined as follows:

```
namespace std {  
    namespace meta {  
        using info = decltype(^int);  
    }  
}
```

In our initial proposal a value of type `std::meta::info` can represent:

- any (C++) type and type alias
- any function or member function
- any variable, static data member, or structured binding
- any non-static data member
- any constant value
- any template
- any namespace

Notably absent at this time are general non-constant expressions (that aren't *expression-ids* referring to functions, variables or structured bindings). For example:

```
int x = 0;  
void g() {  
    [:^(x):] = 42;    // Okay. Same as: x = 42;  
    x = [:^(2*x):];  // Error: "2*x" is a general non-constant expression.  
    constexpr int N = 42;  
    x = [:^(2*N):];  // Okay: "2*N" is a constant-expression.  
}
```

Note that for `:^(2*N)` an implementation only has to capture the constant value of `2*N` and not various other properties of the underlying expression (such as any temporaries it involves, etc.).

The type `std::meta::info` is a *scalar* type. Nontype template arguments of type `std::meta::info` are permitted. The entity being reflected can affect the linkage of a template instance involving a reflection. For example:

```
template<auto R> struct S {};

extern int x;
static int y;

S<^x> sx; // S<^x> has external name linkage.
S<^y> sy; // S<^y> has internal name linkage.
```

Namespace `std::meta` is associated with type `std::meta::info`. That allows the core meta functions to be invoked without explicit qualification. For example:

```
#include <meta>
struct S {};
std::string name2 = std::meta::name_of(^S); // Okay.
std::string name1 = name_of(^S);           // Also okay.
```

§ 4.4 Metafunctions

We propose a number of metafunctions declared in namespace `std::meta` to operator on reflection values. Adding metafunctions to an implementation is expected to be relatively “easy” compared to implementing the core language features described previously. However, despite offering a normal consteval C++ function interface, each on of these relies on “compiler magic” to a significant extent.

§ 4.4.1 Constant evaluation order

In C++23, “constant evaluation” produces pure values without observable side-effects and thus the order in which constant-evaluation occurs is immaterial. In fact, while the language is designed to permit constant evaluation to happen at compile time, an implementation is not strictly required to take advantage of that possibility.

Some of the proposed metafunctions, however, have side-effects that have an effect of the remainder of the program. For example, we provide a `define_class` metafunction that provides a definition for a given class. Clearly, we want the effect of calling that metafunction to be “prompt” in a lexical-order sense. For example:

```
#include <meta>
struct S;

void g() {
    static_assert(is_type(define_class(^S, {})));
    S s; // S should be defined at this point.
}
```

Hence this proposal also introduces constraints on constant evaluation as follows...

First, we identify a subset of manifestly constant-evaluated expressions and conversions characterized by the fact that their evaluation must occur and must succeed in a valid C++ program: We call these *plainly constant-evaluated*. We require that a programmer can count on those evaluations occurring exactly once and completing at translation time.

Second, we sequence plainly constant-evaluated expressions and conversions within the lexical order. Specifically, we require that the evaluation of a plainly constant-evaluated expression or conversion occurs before the implementation checks the validity of source constructs lexically following that expression or conversion.

Those constraints are mostly intuitive, but they are a significant change to the underlying principles of the current standard in this respect.

[P2758R1] (“Emitting messages at compile time”) also has to deal with side effects during constant evaluation. However, those effects (“output”) are of a slightly different nature in the sense that they can be buffered until a

manifestly constant-evaluated expression/conversion has completed. “Buffering” a class type completion is not practical (e.g., because other metafunctions may well depend on the completed class type). Still, we are not aware of incompatibilities between our proposal and [P2758R1].

§ 4.4.2 Error-Handling in Reflection

One important question we have to answer is: How do we handle errors in reflection metafunctions? For example, what does `std::meta::template_of(^int)` do? `^int` is a reflection of a type, but that type is not a specialization of a template, so there is no valid reflected template for us to return.

There are a few options available to us today:

1. This fails to be a constant expression (unspecified mechanism).
2. This returns an invalid reflection (similar to NaN for floating point) which carries source location info and some useful message. (This was the approach suggested in P1240.)
3. This returns `std::expected<std::meta::info, E>` for some reflection-specific error type `E` which carries source location info and some useful message (this could be just `info` but probably should not be).
4. This throws an exception of type `E` (which requires allowing exceptions to work during `constexpr` evaluation, such that an uncaught exception would fail to be a constant expression).

The immediate downside of (2), yielding a NaN-like reflection for `template_of(^int)` is what we do for those functions that need to return a range. That is, what does `template_arguments_of(^int)` return?

1. This fails to be a constant expression (unspecified mechanism).
2. This returns a `std::vector<std::meta::info>` containing one invalid reflection.
3. This returns a `std::expected<std::vector<std::meta::info>, E>`.
4. This throws an exception of type `E`.

Having range-based functions return a single invalid reflection would make for awkward error handling code. Using `std::expected` or exceptions for error handling allow for a consistent, more straightforward interface.

This becomes another situation where we need to decide an error handling mechanism between exceptions and not exceptions, although importantly in this context a lot of usual concerns about exceptions do not apply:

- there is no runtime (so concerns about runtime performance, object file size, etc. do not exist), and
- there is no runtime (so concerns about code evolving to add a new uncaught exception type do not apply)

There is one interesting example to consider to decide between `std::expected` and exceptions here:

```
template <typename T>
requires (template_of(^T) == ^std::optional)
void foo();
```

If `template_of` returns an `expected<info, E>`, then `foo<int>` is a substitution failure — `expected<T, E>` is equality-comparable to `T`, that comparison would evaluate to `false` but still be a constant expression.

If `template_of` returns `info` but throws an exception, then `foo<int>` would cause that exception to be uncaught, which would make the comparison not a constant expression. This actually makes the constraint ill-formed - not a substitution failure. In order to have `foo<int>` be a substitution failure, either the constraint would have to first check that `T` is a template or we would have to change the language rule that requires constraints to be constant expressions (we would of course still keep the requirement that the constraint is a `bool`).

The other thing to consider are compiler modes that disable exception support (like `-fno-exceptions` in GCC and Clang). Today, implementations reject using `try`, `catch`, or `throw` at all when such modes are enabled. With support for `constexpr` exceptions, implementations would have to come up with a strategy for how to support compile-time exceptions — probably by only allowing them in `constexpr` functions (including `constexpr` function templates that were propagated to `constexpr`).

Despite these concerns (and the requirement of a whole new language feature), we believe that exceptions will be the more user-friendly choice for error handling here, simply because exceptions are more ergonomic to use than `std::expected` (even if we adopt language features that make this type easier to use - like pattern matching and a control flow operator).

§ 4.4.3 Handling Aliases

Consider

```
using A = int;
```

In C++ today, `A` and `int` can be used interchangeably and there is no distinction between the two types. With reflection as proposed in this paper, that will no longer be the case. `^A` yields a reflection of an alias to `int`, while `^int` yields a reflection of `int`. `^A == ^int` evaluates to `false`, but there will be a way to strip aliases - so `dealias(^A) == ^int` evaluates to `true`.

This opens up the question of how various other metafunctions handle aliases and it is worth going over a few examples:

```
using A = int;
using B = std::unique_ptr<int>;
template <class T> using C = std::unique_ptr<T>;
```

This paper is proposing that:

- `is_type(^A)` is `true`. `^A` is an alias, but it's an alias to a type, and if this evaluated as `false` then everyone would have to `dealias` everything all the time.
- `has_template_arguments(^B)` is `false` while `has_template_arguments(^C<int>)` is `true`. Even though `B` is an alias to a type that itself has template arguments (`unique_ptr<int>`), `B` itself is simply a type alias and does not. This reflects the actual usage.
- Meanwhile, `template_arguments_of(^C<int>)` yields `{^int}` while `template_arguments_of(^std::unique_ptr<int>)` yields `{^int, ^std::default_deleter<int>}`. This is `C` has its own template arguments that can be reflected on.

§ 4.4.4 Freestanding implementations

Several important metafunctions, such as `std::meta::_nonstatic_data_members_of`, return a `std::vector` value. Unfortunately, that means that they are currently not usable in a freestanding environment. That is an highly undesirable limitation that we believe should be addressed by imbuing freestanding implementations with a more restricted `std::vector` (e.g., one that can only allocate at compile time).

§ 4.4.5 Synopsis

Here is a synopsis for the proposed library API. The functions will be explained below.

```
namespace std::meta {
    // name and location
    constexpr auto name_of(info r) -> string_view;
```

```

constexpr auto qualified_name_of(info r) -> string_view;
constexpr auto display_name_of(info r) -> string_view;
constexpr auto source_location_of(info r) -> source_location;

// type queries
constexpr auto type_of(info r) -> info;
constexpr auto parent_of(info r) -> info;
constexpr auto dealias(info r) -> info;

// template queries
constexpr auto template_of(info r) -> info;
constexpr auto template_arguments_of(info r) -> vector<info>;

// member queries
template<typename ...Fs>
constexpr auto members_of(info class_type, Fs ...filters) -> vector<info>;
template<typename ...Fs>
constexpr auto bases_of(info class_type, Fs ...filters) -> vector<info>;
constexpr auto static_data_members_of(info class_type) -> vector<info>;
constexpr auto nonstatic_data_members_of(info class_type) -> vector<info>;
constexpr auto subobjects_of(info class_type) -> vector<info>;
constexpr auto enumerators_of(info enum_type) -> vector<info>;

// substitute
constexpr auto substitute(info templ, span<info const> args) -> info;

// reflect invoke
constexpr auto reflect_invoke(info target, span<info const> args) -> info;

// value of
template<typename T>
constexpr auto value_of(info) -> T;

// test type
constexpr auto test_type(info templ, info type) -> bool;
constexpr auto test_types(info templ, span<info const> types) -> bool;

// other type predicates \(see the wording\)
constexpr auto is_public(info r) -> bool;
constexpr auto is_protected(info r) -> bool;
constexpr auto is_private(info r) -> bool;
constexpr auto is_accessible(info r) -> bool;
constexpr auto is_virtual(info r) -> bool;
constexpr auto is_pure_virtual(info entity) -> bool;
constexpr auto is_override(info entity) -> bool;
constexpr auto is_deleted(info entity) -> bool;
constexpr auto is_defaulted(info entity) -> bool;
constexpr auto is_explicit(info entity) -> bool;
constexpr auto is_bit_field(info entity) -> bool;
constexpr auto has_static_storage_duration(info r) -> bool;
constexpr auto has_internal_linkage(info r) -> bool;
constexpr auto has_external_linkage(info r) -> bool;
constexpr auto has_linkage(info r) -> bool;
constexpr auto is_class_member(info entity) -> bool;
constexpr auto is_namespace_member(info entity) -> bool;
constexpr auto is_nonstatic_data_member(info entity) -> bool;
constexpr auto is_static_member(info entity) -> bool;
constexpr auto is_base(info entity) -> bool;
constexpr auto is_namespace(info entity) -> bool;
constexpr auto is_function(info entity) -> bool;
constexpr auto is_variable(info entity) -> bool;
constexpr auto is_type(info entity) -> bool;

```

```

constexpr auto is_alias(info entity) -> bool;
constexpr auto is_incomplete_type(info entity) -> bool;
constexpr auto is_template(info entity) -> bool;
constexpr auto is_function_template(info entity) -> bool;
constexpr auto is_variable_template(info entity) -> bool;
constexpr auto is_class_template(info entity) -> bool;
constexpr auto is_alias_template(info entity) -> bool;
constexpr auto is_concept(info entity) -> bool;
constexpr auto has_template_arguments(info r) -> bool;
constexpr auto is_constructor(info r) -> bool;
constexpr auto is_destructor(info r) -> bool;
constexpr auto is_special_member(info r) -> bool;

// reflect value
template<typename T>
constexpr auto reflect_value(T value) -> info;

// define class
struct data_member_options_t;
constexpr auto data_member_spec(info class_type,
                                data_member_options_t options = {}) -> info;
constexpr auto define_class(info class_type, span<info const>) -> info;

// data layout
constexpr auto offset_of(info entity) -> size_t;
constexpr auto size_of(info entity) -> size_t;
constexpr auto bit_offset_of(info entity) -> size_t;
constexpr auto bit_size_of(info entity) -> size_t;
constexpr auto alignment_of(info entity) -> size_t;
}

```

§ 4.4.6 name_of, display_name_of, source_location_of

```

namespace std::meta {
constexpr auto name_of(info r) -> string_view;
constexpr auto qualified_name_of(info r) -> string_view;
constexpr auto display_name_of(info r) -> string_view;
constexpr auto source_location_of(info r) -> source_location;
}

```

Given a reflection *r* that designates a declared entity *X*, *name_of(r)* and *qualified_name_of(r)* return a *string_view* holding the unqualified and qualified name of *X*, respectively. For all other reflections, an empty *string_view* is produced. For template instances, the name does not include the template argument list. The contents of the *string_view* consist of characters of the basic source character set only (an implementation can map other characters using universal character names).

Given a reflection *r*, *display_name_of(r)* returns an unspecified non-empty *string_view*. Implementations are encouraged to produce text that is helpful in identifying the reflected construct.

Given a reflection *r*, *source_location_of(r)* returns an unspecified *source_location*. Implementations are encouraged to produce the correct source location of the item designated by the reflection.

§ 4.4.7 type_of, parent_of, dealias

```

namespace std::meta {
constexpr auto type_of(info r) -> info;
constexpr auto parent_of(info r) -> info;
constexpr auto dealias(info r) -> info;
}

```

If `r` is a reflection designating a typed entity, `type_of(r)` is a reflection designating its type. If `r` is already a type, `type_of(r)` is not a constant expression. This can be used to implement the C `typeof` feature (which works on both types and expressions and strips qualifiers):

```
constexpr auto do_typeof(std::meta::info r) -> std::meta::info {
    return remove_cvref(is_type(r) ? r : type_of(r));
}

#define typeof(e) [: do_typeof(^e) :]
```

If `r` designates a member of a class or namespace, `parent_of(r)` is a reflection designating its immediately enclosing class or namespace.

If `r` designates an alias, `dealias(r)` designates the underlying entity. Otherwise, `dealias(r)` produces `r`. `dealias` is recursive - it strips all aliases:

```
using X = int;
using Y = X;
static_assert(dealias(^int) == ^int);
static_assert(dealias(^X) == ^int);
static_assert(dealias(^Y) == ^int);
```

§ 4.4.8 `template_of`, `template_arguments_of`

```
namespace std::meta {
    constexpr auto template_of(info r) -> info;
    constexpr auto template_arguments_of(info r) -> vector<info>;
}
```

If `r` is a reflection designated a specialization of some template, then `template_of(r)` is a reflection of that template and `template_arguments_of(r)` is a vector of the reflections of the template arguments. In other words, the preconditions on both is that `has_template_arguments(r)` is `true`.

For example:

```
std::vector<int> v = {1, 2, 3};
static_assert(template_of(type_of(^v)) == ^std::vector);
static_assert(template_arguments_of(type_of(^v))[0] == ^int);
```

§ 4.4.9 `members_of`, `static_data_members_of`, `nonstatic_data_members_of`, `bases_of`, `enumerators_of`, `subobjects_of`

```
namespace std::meta {
    template<typename ...Fs>
        constexpr auto members_of(info class_type, Fs ...filters) -> vector<info>;

    template<typename ...Fs>
        constexpr auto bases_of(info class_type, Fs ...filters) -> vector<info>;

    constexpr auto static_data_members_of(info class_type) -> vector<info> {
        return members_of(class_type, is_variable);
    }

    constexpr auto nonstatic_data_members_of(info class_type) -> vector<info> {
        return members_of(class_type, is_nonstatic_data_member);
    }

    constexpr auto subobjects_of(info class_type) -> vector<info> {
        auto subobjects = bases_of(class_type);
    }
```

```

    subobjects.append_range(nonstatic_data_members_of(class_type));
    return subobjects;
}

constexpr auto enumerators_of(info enum_type) -> vector<info>;
}

```

The template `members_of` returns a vector of reflections representing the direct members of the class type represented by its first argument. Any nonstatic data members appear in declaration order within that vector. Anonymous unions appear as a nonstatic data member of corresponding union type. If any `Filters...` argument is specified, a member is dropped from the result if any filter applied to that members reflection returns `false`. E.g., `members_of(^C, std::meta::is_type)` will only return types nested in the definition of `C` and `members_of(^C, std::meta::is_type, std::meta::is_variable)` will return an empty vector since a member cannot be both a type and a variable.

The template `bases_of` returns the direct base classes of the class type represented by its first argument, in declaration order.

`enumerators_of` returns the enumerator constants of the indicated enumeration type in declaration order.

§ 4.4.10 substitute

```

namespace std::meta {
    constexpr auto substitute(info templ, span<info const> args) -> info;
}

```

Given a reflection for a template and reflections for template arguments that match that template, `substitute` returns a reflection for the entity obtained by substituting the given arguments in the template. If the template is a concept template, the result is a reflection of a constant of type `bool`.

For example:

```

constexpr auto r = substitute(^std::vector, std::vector{^int});
using T = [:r:]; // Ok, T is std::vector<int>

```

This process might kick off instantiations outside the immediate context, which can lead to the program being ill-formed.

Note that the template is only substituted, not instantiated. For example:

```

template<typename T> struct S { typename T::X x; };

constexpr auto r = substitute(^S, std::vector{^int}); // Okay.
typename[:r:] si; // Error: T::X is invalid for T = int.

```

§ 4.4.11 reflect_invoke

```

namespace std::meta {
    constexpr auto reflect_invoke(info target, span<info const> args) -> info;
}

```

This metafunction produces a reflection of the value returned by a call expression.

Letting `F` be the entity reflected by `target`, and `A0, ..., AN` be the sequence of entities reflected by the values held by `args`: if the expression `F(A0, ..., AN)` is a well-formed constant expression evaluating to a type that is not `void`, and if every value in `args` is a reflection of a constant value, then `reflect_invoke(target, args)` evaluates to a reflection of the constant value `F(A0, ..., AN)`.

For all other invocations, `reflect_invoke(target, args)` is ill-formed.

§ 4.4.12 `value_of<T>`

```
namespace std::meta {  
    template<typename T> consteval auto value_of(info) -> T;  
}
```

If `r` is a reflection for a constant-expression or a constant-valued entity of type `T`, `value_of<T>(r)` evaluates to that constant value.

If `r` is a reflection for a variable of non-reference type `T`, `value_of<T&>(r)` and `value_of<T const&>(r)` are lvalues referring to that variable. If the variable is usable in constant expressions [expr.const], `value_of<T>(r)` evaluates to its value.

If `r` is a reflection for a variable of reference type `T` usable in constant-expressions, `value_of<T>(r)` evaluates to that reference.

If `r` is a reflection of an enumerator constant of type `E`, `value_of<E>(r)` evaluates to the value of that enumerator.

If `r` is a reflection of a non-bit-field non-reference non-static member of type `M` in a class `C`, `value_of<M C::*>(r)` is the pointer-to-member value for that nonstatic member.

For other reflection values `r`, `value_of<T>(r)` is ill-formed.

The function template `value_of` may feel similar to splicers, but unlike splicers it does not require its operand to be a constant-expression itself. Also unlike splicers, it requires knowledge of the type associated with the entity reflected by its operand.

§ 4.4.13 `test_type`, `test_types`

```
namespace std::meta {  
    consteval auto test_type(info templ, info type) -> bool {  
        return test_types(templ, {type});  
    }  
  
    consteval auto test_types(info templ, span<info const> types) -> bool {  
        return value_of<bool>(substitute(templ, types));  
    }  
}
```

This utility translates existing metaprogramming predicates (expressed as `constexpr` variable templates or concept templates) to the reflection domain. For example:

```
struct S {};  
static_assert(test_type(^std::is_class_v, ^S));
```

An implementation is permitted to recognize standard predicate templates and implement `test_type` without actually instantiating the predicate template. In fact, that is recommended practice.

§ 4.4.14 `reflect_value`

```
namespace std::meta {  
    template<typename T> consteval auto reflect_value(T value) -> info;  
}
```

This metafunction produces a reflection representing the constant value of the operand.

§ 4.4.15 data_member_spec, define_class

```
namespace std::meta {
    struct data_member_options_t {
        optional<string_view> name;
        bool is_static = false;
        optional<int> alignment;
        optional<int> width;
    };
    consteval auto data_member_spec(info type,
                                    data_member_options_t options = {}) -> info;
    consteval auto define_class(info class_type, span<info const>) -> info;
}
```

`data_member_spec` returns a reflection of a description of a data member of given type. Optional alignment, bit-field-width, static-ness, and name can be provided as well. If no name is provided, the name of the data member is unspecified. If `is_static` is `true`, the data member is declared `static`.

`define_class` takes the reflection of an incomplete class/struct/union type and a range of reflections of data member descriptions and it completes the given class type with data members as described (in the given order). The given reflection is returned. For now, only data member reflections are supported (via `data_member_spec`) but the API takes in a range of `info` anticipating expanding this in the near future.

For example:

```
union U;
static_assert(is_type(define_class(^U, {
    data_member_spec(^int),
    data_member_spec(^char),
    data_member_spec(^double),
})));

// U is now defined to the equivalent of
// union U {
//     int _0;
//     char _1;
//     double _2;
// };

template<typename T> struct S;
constexpr auto U = define_class(^S<int>, {
    data_member_spec(^int, {.name="i", .align=64}),
    data_member_spec(^int, {.name="j", .align=64}),
});

// S<int> is now defined to the equivalent of
// template<> struct S<int> {
//     alignas(64) int i;
//     alignas(64) int j;
// };

```

When defining a `union`, if one of the alternatives has a non-trivial destructor, the defined union will *still* have a destructor provided - that simply does nothing. This allows implementing [variant](#) without having to further extend support in `define_class` for member functions.

§ 4.4.16 Data Layout Reflection

```
namespace std::meta {
    consteval auto offset_of(info entity) -> size_t;
```

```

consteval auto size_of(info entity) -> size_t;

consteval auto bit_offset_of(info entity) -> size_t;
consteval auto bit_size_of(info entity) -> size_t;

consteval auto alignment_of(info entity) -> size_t;
}

```

§ 4.4.17 Other Type Traits

There is a question of whether all the type traits should be provided in `std::meta`. For instance, a few examples in this paper use `std::meta::remove_cvref(t)` as if that exists. Technically, the functionality isn't strictly necessary - since it can be provided indirectly:

Direct	Indirect
<code>std::meta::remove_cvref(type)</code>	<code>std::meta::substitute(^std::remove_cvref_t, {type})</code>
<code>std::meta::is_const(type)</code>	<code>std::meta::value_of<bool></code> <code>(std::meta::substitute(^std::is_const_v, {type}))</code> <code>std::meta::test_type(^std::is_const_v, type)</code>

Having `std::meta::meow` for every trait `std::meow` is more straightforward and will likely be faster to compile, though means we will have a much larger library API. There are quite a few traits in 21 [\[meta\]](#) - but it should be easy enough to specify all of them. So we're doing it.

§ 5 Proposed Wording

§ 5.1 Language

§ 5.1.1 [lex.phases] Phases of translation

Modify the wording for phases 7-8 of 5.2 [\[lex.phases\]](#) as follows:

- 7 Whitespace characters separating tokens are no longer significant. Each preprocessing token is converted into a token (5.6). The resulting tokens constitute a translation unit and are syntactically and semantically analyzed and translated. Plainly constant-evaluated expressions (`[expr.const]`) appearing outside template declarations are evaluated in lexical order. Diagnosable rules (4.1.1 [\[intro.compliance.general\]](#)) that apply to constructs whose syntactic end point occurs lexically after the syntactic end point of a plainly constant-evaluated expression X are considered in a context where X has been evaluated. [...]
- 8 [...] All the required instantiations are performed to produce instantiation units. Plainly constant-evaluated expressions (`[expr.const]`) appearing in those instantiation units are evaluated in lexical order as part of the instantiation process. Diagnosable rules (4.1.1 [\[intro.compliance.general\]](#)) that apply to constructs whose syntactic end point occurs lexically after the syntactic end point of a plainly constant-evaluated expression X are considered in a context where X has been evaluated. [...]

§ 5.1.2 [lex.pptoken] Preprocessing tokens

Add a bullet after 5.4 [\[lex.pptoken\]](#) bullet (3.2):

...

— Otherwise, if the next three characters are `<::` and the subsequent character is neither `:` nor `>`, the `<` is treated as a preprocessing token by itself and not as the first character of the alternative token `<:`.

— Otherwise, if the next three characters are `[::` and the subsequent character is not `:`, the `[` is treated as a preprocessing token by itself and not as the first character of the preprocessing token `[:`.

...

§ 5.1.3 [lex.operators] Operators and punctuators

Change the grammar for *operator-or-punctuator* in paragraph 1 of 5.12 [lex.operators] to include splicer delimiters:

```
operator-or-punctuator: one of
    {      }      [      ]      (      )      [:::]
    <:      :>      <%      %>      ;      :      ...
    ?      ::      .      .*      ->      ->*      ~
    !      +      -      *      /      %      ^      &      |
    =      +=      -=      *=      /=      %=      ^=      &=      |=
    ==     !=     <     >     <=     >=     <=>     &&     ||
    <<     >>     <<=    >>=    ++     --     ,      compl
    and    or     xor    not    bitand  bitor   '      compl
    and_eq or_eq  xor_eq not_eq
```

§ 5.1.4 [basic.types.general]

Change the first sentence in paragraph 9 of 6.8.1 [basic.types.general] as follows:

- 9 Arithmetic types (6.8.2), enumeration types, pointer types, pointer-to-member types (6.8.4), `std::meta::info`, `std::nullptr_t`, and cv-qualified (6.8.5) versions of these types are collectively called scalar types.

Add a new paragraph at the end of 6.8.1 [basic.types.general] as follows:

- * A *consteval-only type* is one of the following:
- `std::meta::info`, or
 - a pointer or reference to a consteval-only type, or
 - an (possibly multi-dimensional) array of a consteval-only type, or
 - a pointer-to-member type to a class *C* of type *M* where either *C* or *M* is a consteval-only type, or
 - a function type with a consteval-only return type or a consteval-only parameter type, or
 - a class type with a consteval-only base class type or consteval-only non-static data member type.

An object of consteval-only type shall either end its lifetime during the evaluation of a manifestly constant-evaluated expression or conversion (7.7 [expr.const]), or be a constexpr variable that is not odr-used (6.3 [basic.def.odr]).

§ 5.1.5 [basic.fundamental] Fundamental types

Add a new paragraph before the last paragraph of 6.8.2 [basic.fundamental] as follows:

- * A value of type `std::meta::info` is called a *reflection* and represents a language element such as a type, a constant value, a non-static data member, etc. An expression convertible to `std::meta::info` is said to

reflect the language element represented by the resulting value; the language element is said to be reflected by the expression. `sizeof(std::meta::info)` shall be equal to `sizeof(void*)`. [Note 1: Reflections are only meaningful during translation. The notion of consteval-only types (see 6.8.1 [basic.types.general]) exists to diagnose attempts at using such values outside the translation process. — end note]

§ 5.1.6 [basic.lookup.argdep] Argument-dependent name lookup

Add a bullet after the first in paragraph 3 of [basic.lookup.argdep] as follows:

- 3 ... Any *typedef-names* and *using-declarations* used to specify the types do not contribute to this set. The set of entities is determined in the following way:

- (3.1) — If τ is a fundamental type, its associated set of entities is empty.
- (3.2) — If τ is `std::meta::info`, its associated set of entities is the singleton containing the function `std::meta::is_type`.
- (3.3) — If τ is a class type ...

§ 5.1.7 [basic.lookup.qual.general] General

Extend 6.5.5.1 [basic.lookup.qual.general]/1-2 to cover *splice-name-qualifier*:

- 1 Lookup of an *identifier* followed by a `::` scope resolution operator considers only namespaces, types, and templates whose specializations are types. If a name, *template-id*, ~~*or computed-type-specifier*~~, or *splice-name-qualifier* is followed by a `::`, it shall designate a namespace, class, enumeration, or dependent type, and the `::` is never interpreted as a complete nested-name-specifier.
- 2 A member-qualified name is the (unique) component name ([*expr.prim.id.unqual*]), if any, of
 - (2.1) — an *unqualified-id* or
 - (2.2) — a *nested-name-specifier* of the form *type-name* `::` ~~*or*~~, *namespace-name* `::`, or *splice-name-qualifier* `::`

in the *id-expression* of a class member access expression ([*expr.ref*]). [...]

§ 5.1.8 [expr.prim] Primary expressions

Change the grammar for *primary-expression* in 7.5 [expr.prim] as follows:

```
primary-expression:
    literal
    this
    ( expression )
    id-expression
    lambda-expression
    fold-expression
    requires-expression
+   [: constant-expression :]
+   template[: constant-expression :] < template-argument-listopt >
```

§ 5.1.9 [expr.prim.id.qual] Qualified names

Add a production to the grammar for *nested-name-specifier* as follows:

```

    nested-name-specifier:
        ::
        type-name ::
        namespace-name ::
        computed-type-specifier ::
+   splice-name-qualifier ::
    nested-name-specifier identifier ::
    nested-name-specifier templateopt simple-template-id ::
+
+   splice-name-qualifier:
+   [: constant-expression :]

```

Extend 7.5.4.3 [\[expr.prim.id.qual\]](#)/1 to also cover splices:

- 1 The component names of a *qualified-id* are those of its *nested-name-specifier* and *unqualified-id*. The component names of a *nested-name-specifier* are its *identifier* (if any) and those of its *type-name*, *namespace-name*, *simple-template-id*, ~~and/or~~ *nested-name-specifier*, and/or the *type-name* or *namespace-name* of the entity reflected by the *constant-expression* of its *splice-name-qualifier*. For a *nested-name-specifier* having a *splice-name-qualifier* with a *constant-expression* that reflects the global namespace, the component names are the same as for `::`. The *constant-expression* of a *splice-name-qualifier* shall be a reflection of either a *type-name*, *namespace-name*, or the global namespace.

Extend 7.5.4.3 [\[expr.prim.id.qual\]](#)/3 to also cover splices:

- 3 The *nested-name-specifier* `::` nominates the global namespace. A *nested-name-specifier* with a *computed-type-specifier* nominates the type denoted by the *computed-type-specifier*, which shall be a class or enumeration type. A *nested-name-specifier* with a *splice-name-qualifier* nominates the entity reflected by the *constant-expression* of the *splice-name-qualifier*. If a *nested-name-specifier* N is declarative and has a *simple-template-id* with a template argument list A that involves a template parameter, let T be the template nominated by N without A. T shall be a class template.

...

§ 5.1.10 [\[expr.prim.splice\]](#) Expression splicing

Add a new subsection of 7.5 [\[expr.prim\]](#) following 7.5.7 [\[expr.prim.req\]](#)

Expression Splicing [\[expr.prim.splice\]](#)

- 1 For a *primary-expression* of the form `[: constant-expression :]` or `template[: constant-expression :] < template-argument-listopt >` the *constant-expression* shall be a converted constant expression (7.7 [\[expr.const\]](#)) of type `std::meta::info`.
- 2 For a *primary-expression* of the form `template[: constant-expression :] < template-argument-listopt >` the converted *constant-expression* shall evaluate to a reflection for a concept, variable template, class template, alias template, or function template. The meaning of such a construct is identical to that of a *primary-expression* of the form `template-name < template-argument-listopt >` where *template-name* denotes the reflected template or concept (ignoring access checking on the *template-name*).
- 3 For a *primary-expression* of the form `[: constant-expression :]` where the converted *constant-expression* evaluates to a reflection for a variable, a function, an enumerator, or a structured binding, the

meaning of the expression is identical to that of a *primary-expression* of the form *id-expression* that would denote the reflected entity (ignoring access checking).

- 4 Otherwise, for a *primary-expression* of the form `[ : : constant-expression : ]` the converted *constant-expression* shall evaluate to a reflection for a constant value and the expression shall evaluate to that value.

§ 5.1.11 [expr.unary.general]

Change 7.6.2.1 [expr.unary.general] paragraph 1 to add productions for the new operator:

- 1 Expressions with unary operators group right-to-left.

```
unary-expression:
    ...
    delete-expression
+   ^ ::
+   ^ namespace-name
+   ^ nested-name-specifieropt template-name
+   ^ nested-name-specifieropt concept-name
+   ^ type-id
+   ^ cast-expression
```

§ 5.1.12 [expr.reflect] The reflection operator

Add a new subsection of 7.6.2 [expr.unary] following 7.6.2.9 [expr.delete]

The Reflection Operator [expr.reflect]

- 1 The unary `^` operator (called *the reflection operator*) produces a prvalue — called *reflection* — whose type is the reflection type (i.e., `std::meta::info`). That reflection represents its operand.
- 2 An ambiguity can arise between the interpretation of the operand of the reflection operator as a *type-id* or a *cast-expression*; in such cases, the *type-id* treatment is chosen. Parentheses can be introduced to force the *cast-expression* interpretation.
- 3 [Example

```
static_assert(is_type(^int()));    // ^ applies to the type-id "int()"; not the cast
    "int()"
static_assert(!is_type(^int())); // ^ applies to the the cast-expression "(int())"

template<bool> struct X;
constexpr void g(std::meta::info r) {
    if (r == ^int && true);    // error: ^ applies to the type-id "int&&"
    if (r == (^int) && true);  // OK
    if (r == ^X < true);      // error: "<" is an angle bracket
    if (r == (^X) < true);    // OK
}
```

-end example]

- 4 When applied to `::`, the reflection operator produces a reflection for the global namespace. When applied to a *namespace-name*, the reflection produces a reflection for the indicated namespace or namespace alias.

- 5 When applied to a *template-name*, the reflection produces a reflection for the indicated template.
- 6 When applied to a *concept-name*, the reflection produces a reflection for the indicated concept.
- 7 When applied to a *type-id*, the reflection produces a reflection for the indicated type or type alias.
- 8 When applied to a *cast-expression*, the *cast-expression* shall be a constant expression (7.7 [expr.const]) or an *id-expression* (7.5.4 [expr.prim.id]) designating a variable, a function, an enumerator constant, or a nonstatic member. The *cast-expression* is not evaluated.
- (8.1) — If the operand of the reflection operator is an *id-expression*, the result is a reflection for the indicated entity.
- (8.1.1) — If this *id-expression* names an overload set S, and if the assignment of S to an invented variable of type `const auto` (9.2.9.6.2 [dcl.type.auto.deduct]) would select a unique candidate function F from S, the result is a reflection of F. Otherwise, the expression `^S` is ill-formed.
- (8.2) — If the operand is a constant expression, the result is a reflection for the resulting value.
- (8.3) — If the operand is both an *id-expression* and a constant expression, the result is a reflection for both the indicated entity and the expression's (constant) value.

[Example:

```
template <typename T> void fn() requires (^T != ^int);
template <typename T> void fn() requires (^T == ^int);
template <typename T> void fn() requires (sizeof(T) == sizeof(int));

constexpr auto R = ^fn<char>;      // OK
constexpr auto S = ^fn<int>;      // error: cannot reflect an overload set

constexpr auto r = ^std::vector;  // OK

— end example ]
```

§ 5.1.13 [expr.eq] Equality Operators

Extend 7.6.10 [expr.eq]/2 to also handle `std::meta::info`:

- 2 The converted operands shall have arithmetic, enumeration, pointer, or pointer-to-member type, or **type** `std::meta::info` or `std::nullptr_t`. The operators `==` and `!=` both yield `true` or `false`, i.e., a result of type `bool`. In each case below, the operands shall have the same type after the specified conversions have been applied.

Add a new paragraph between 7.6.10 [expr.eq]/5 and /6:

- 5 Two operands of type `std::nullptr_t` or one operand of type `std::nullptr_t` and the other a null pointer constant compare equal.
- * If both operands are of type `std::meta::info`, comparison is defined as follows:
- (*.1) — If one operand is a reflection of a namespace alias, alias template, or type alias and the other operand is not a reflection of the same kind of alias, they compare unequal. [Note 1: A reflection of a type and a reflection of an alias to that same type do not compare equal. — end note]
- (*.2) — Otherwise, if both operands are reflections of a namespace alias, alias template, or type alias, then they compare equal if they are reflections of the same namespace alias, alias template, or type alias,

respectively.

- (*.3) — Otherwise, if neither operand is a reflection of an expression, then they compare equal if they are reflections of the same entity.
- (*.4) — Otherwise, if one operand is a reflection of an expression and the other is not, then they compare unequal.
- (*.5) — Otherwise (if both operands are reflections of expressions):
 - (*.5.1) — If both operands designate *id-expressions*, then they compare equal if they identify the same declared entity.
 - (*.5.2) — Otherwise, if one operand designates an *id-expression*, then they compare unequal.
 - (*.5.3) — Otherwise, the result is unspecified.

- 6 If two operands compare equal, the result is `true` for the `==` operator and `false` for the `!=` operator. If two operands compare unequal, the result is `false` for the `==` operator and `true` for the `!=` operator. Otherwise, the result of each of the operators is unspecified.

§ 5.1.14 [expr.const] Constant Expressions

Add a new paragraph after the definition of *manifestly constant-evaluated* 7.7 [expr.const]/20:

- 21 An expression or conversion is *plainly constant-evaluated* if it is:
- (21.1) — a *constant-expression*, or
 - (21.2) — the condition of a `constexpr` if statement (8.5.2 [stmt.if]),
 - (21.3) — the initializer of a `constexpr` (9.2.6 [dcl.constexpr]) or `constexpr` (9.2.7 [dcl.constinit]) variable, or
 - (21.4) — an immediate invocation, unless it
 - (21.4.1) — results from the substitution of template parameters in a concept-id (13.3 [temp.names]), a *requires-expression* (7.5.7 [expr.prim.req]), or during template argument deduction (13.10.3 [temp.deduct]), or
 - (21.4.2) — is a manifestly constant-evaluated initializer of a variable that is neither `constexpr` (9.2.6 [dcl.constexpr]) nor `constexpr` (9.2.7 [dcl.constinit]).

§ 5.1.15 [dcl.typedef] The `typedef` specifier

Introduce the term “type alias” to 9.2.4 [dcl.typedef]:

- 1 [...] A name declared with the `typedef` specifier becomes a typedef-name. A typedef-name names the type associated with the identifier ([dcl.decl]) or simple-template-id ([temp.pre]); a typedef-name is thus a synonym for another type. A typedef-name does not introduce a new type the way a class declaration ([class.name]) or enum declaration ([dcl.enum]) does.
- 2 A *typedef-name* can also be introduced by an alias-declaration. The identifier following the `using` keyword is not looked up; it becomes a typedef-name and the optional attribute-specifier-seq following the identifier appertains to that typedef-name. Such a typedef-name has the same semantics as if it were introduced by the typedef specifier. In particular, it does not define a new type.

* A *type alias* is either a name declared with the `typedef` specifier or a name introduced by an *alias-declaration*.

§ 5.1.16 [dcl.attr.grammar] Attribute syntax and semantics

Add a production to the grammar for *attribute-specifier* as follows:

```
attribute-specifier:  
  [ [ attribute-using-prefixopt attribute-list ] ]  
+ [ [ using attribute-namespace : ] ]  
  alignment-specifier
```

and update the grammar for *balanced token* as follows:

```
balanced-token :  
  ( balanced-token-seqopt )  
  [ balanced-token-seqopt ]  
  { balanced-token-seqopt }  
- any token other than a parenthesis, a bracket, or a brace  
+ [ : balanced-token-seqopt : ]  
+ any token other than (, ), [, ], {, }, [ :, or : ]
```

Change a sentence in paragraph 4 of 9.12.1 [dcl.attr.grammar] as follows:

- 4 [...] An *attribute-specifier* that contains no *attributes* and no *alignment-specifier* has no effect.
[Note 1: That includes an *attribute-specifier* of the form [[using attribute-namespace :]] which is thus equivalent to replacing the :] token by the two-token sequence :]. — end note] ...

§ 5.1.17 [temp.names] Names of template specializations

Modify the grammar for *template-argument* as follows:

```
+ splice-template-argument:  
+ [ : constant-expression : ]  
+  
  template-argument:  
    constant-expression  
    type-id  
    id-expression  
    braced-init-list  
+ splice-template-argument
```

§ 5.1.18 [temp.arg.general] General

Adjust paragraph 3 of [temp.arg.general] to not apply to splice template arguments:

- 3 In a *template-argument* which does not contain a *splice-template-argument*, an ambiguity between a *type-id* and an expression is resolved to a *type-id*, regardless of the form of the corresponding *template-parameter*. In a *template-argument* containing a *splice-template-argument*, an ambiguity between a *splice-template-argument* and an expression is resolved to a *splice-template-argument*.

§ 5.1.19 [temp.arg.type] Template type arguments

Extend 13.4.2 [temp.arg.type]/1 to cover splice template arguments:

- 1 A *template-argument* for a *template-parameter* which is a type shall either be a *type-id* or a *splice-template-argument*. A *template-argument* having a *splice-template-argument* for such a *template-parameter* is treated as if were a *type-id* nominating the type reflected by the *constant-expression* of the *splice-template-argument*.

§ 5.1.20 [temp.arg.nontype] Template non-type arguments

Extend 13.4.3 [temp.arg.nontype]/2 to cover splice template arguments:

- 2 The value of a non-type *template-parameter* P of (possibly deduced) type T is determined from its template argument A as follows. If T is not a class type and A is ~~not~~ neither a *braced-init-list* nor a *splice-template-argument*, A shall be a converted constant expression ([expr.const]) of type T ; the value of P is A (as converted).

§ 5.1.21 [temp.arg.template] Template template arguments

Extend 13.4.4 [temp.arg.template]/1 to cover splice template arguments:

- 1 A *template-argument* for a template *template-parameter* shall be the name of a class template or an alias template, expressed as *id-expression*, or a *splice-template-argument*. A *template-argument* for a template *template-parameter* having a *splice-template-argument* is treated as an *id-expression* nominating the class template or alias template reflected by the *constant-expression* of the *splice-template-argument*.

§ 5.2 Library

§ 5.2.1 [over.built] Built-in operators

Add built-in operator candidates for `std::meta::info` to 12.5 [over.built]:

- 16 For every T , where T is a pointer-to-member type, `std::meta::info`, or `std::nullptr_t`, there exist candidate operator functions of the form

```
bool operator==(T, T);
bool operator!=(T, T);
```

§ 5.2.2 Header <meta> synopsis

Add a new subsection in 21 [meta] after 21.3 [type.traits]:

Header <meta> synopsis

```
namespace std::meta {
    using info = decltype(^::);

    // [meta.reflection.names], reflection names and locations
    consteval string_view name_of(info r);
    consteval string_view qualified_name_of(info r);
    consteval string_view display_name_of(info r);
    consteval source_location source_location_of(info r);

    // [meta.reflection.queries], reflection queries
    consteval bool is_public(info r);
    consteval bool is_protected(info r);
    consteval bool is_private(info r);
    consteval bool is_accessible(info r);
    consteval bool is_virtual(info r);
    consteval bool is_pure_virtual(info r);
    consteval bool is_override(info r);
    consteval bool is_deleted(info r);
    consteval bool is_defaulted(info r);
    consteval bool is_explicit(info r);
```

```

constexpr bool is_bit_field(info r);
constexpr bool has_static_storage_duration(info r);
constexpr bool has_internal_linkage(info r);
constexpr bool has_external_linkage(info r);
constexpr bool has_linkage(info r);

constexpr bool is_namespace(info r);
constexpr bool is_function(info r);
constexpr bool is_variable(info r);
constexpr bool is_type(info r);
constexpr bool is_alias(info r);
constexpr bool is_incomplete_type(info r);
constexpr bool is_template(info r);
constexpr bool is_function_template(info r);
constexpr bool is_variable_template(info r);
constexpr bool is_class_template(info r);
constexpr bool is_alias_template(info r);
constexpr bool is_concept(info r);
constexpr bool has_template_arguments(info r);
constexpr auto is_class_member(info entity) -> bool;
constexpr auto is_namespace_member(info entity) -> bool;
constexpr bool is_nonstatic_data_member(info r);
constexpr bool is_static_member(info r);
constexpr bool is_base(info r);
constexpr bool is_constructor(info r);
constexpr bool is_destructor(info r);
constexpr bool is_special_member(info r);

constexpr info type_of(info r);
constexpr info parent_of(info r);
constexpr info dealias(info r);
constexpr info template_of(info r);
constexpr vector<info> template_arguments_of(info r);

// [meta.reflection.member.queries], reflection member queries
template<class... Fs>
constexpr vector<info> members_of(info type, Fs... filters);
template<class... Fs>
constexpr vector<info> bases_of(info type, Fs... filters);
constexpr vector<info> static_data_members_of(info type);
constexpr vector<info> nonstatic_data_members_of(info type);
constexpr vector<info> subobjects_of(info type);
constexpr vector<info> enumerators_of(info enum_type);

// [meta.reflection.unary.cat], primary type categories
constexpr bool is_void(info type);
constexpr bool is_null_pointer(info type);
constexpr bool is_integral(info type);
constexpr bool is_floating_point(info type);
constexpr bool is_array(info type);
constexpr bool is_pointer(info type);
constexpr bool is_lvalue_reference(info type);
constexpr bool is_rvalue_reference(info type);
constexpr bool is_member_object_pointer(info type);
constexpr bool is_member_function_pointer(info type);
constexpr bool is_enum(info type);
constexpr bool is_union(info type);
constexpr bool is_class(info type);
constexpr bool is_function(info type);

// [meta.reflection.unary.comp], composite type categories
constexpr bool is_reference(info type);

```

```
constexpr bool is_arithmetic(info type);
constexpr bool is_fundamental(info type);
constexpr bool is_object(info type);
constexpr bool is_scalar(info type);
constexpr bool is_compound(info type);
constexpr bool is_member_pointer(info type);

// [meta.reflection.unary.prop], type properties
constexpr bool is_const(info type);
constexpr bool is_volatile(info type);
constexpr bool is_trivial(info type);
constexpr bool is_trivially_copyable(info type);
constexpr bool is_standard_layout(info type);
constexpr bool is_empty(info type);
constexpr bool is_polymorphic(info type);
constexpr bool is_abstract(info type);
constexpr bool is_final(info type);
constexpr bool is_aggregate(info type);
constexpr bool is_signed(info type);
constexpr bool is_unsigned(info type);
constexpr bool is_bounded_array(info type);
constexpr bool is_unbounded_array(info type);
constexpr bool is_scoped_enum(info type);

constexpr bool is_constructible(info type, span<info const> type_args);
constexpr bool is_default_constructible(info type);
constexpr bool is_copy_constructible(info type);
constexpr bool is_move_constructible(info type);

constexpr bool is_assignable(info dst_type, info src_type);
constexpr bool is_copy_assignable(info type);
constexpr bool is_move_assignable(info type);

constexpr bool is_swappable_with(info dst_type, info src_type);
constexpr bool is_swappable(info type);

constexpr bool is_destructible(info type);

constexpr bool is_trivially_constructible(info type, span<info const> type_args);
constexpr bool is_trivially_default_constructible(info type);
constexpr bool is_trivially_copy_constructible(info type);
constexpr bool is_trivially_move_constructible(info type);

constexpr bool is_trivially_assignable(info dst_type, info src_type);
constexpr bool is_trivially_copy_assignable(info type);
constexpr bool is_trivially_move_assignable(info type);
constexpr bool is_trivially_destructible(info type);

constexpr bool is_nothrow_constructible(info type, span<info const> type_args);
constexpr bool is_nothrow_default_constructible(info type);
constexpr bool is_nothrow_copy_constructible(info type);
constexpr bool is_nothrow_move_constructible(info type);

constexpr bool is_nothrow_assignable(info dst_type, info src_type);
constexpr bool is_nothrow_copy_assignable(info type);
constexpr bool is_nothrow_move_assignable(info type);

constexpr bool is_nothrow_swappable_with(info dst_type, info src_type);
constexpr bool is_nothrow_swappable(info type);

constexpr bool is_nothrow_destructible(info type);
```

```

consteval bool is_implicit_lifetime(info type);

consteval bool has_virtual_destructor(info type);

consteval bool has_unique_object_representations(info type);

consteval bool reference_constructs_from_temporary(info dst_type, info src_type);
consteval bool reference_converts_from_temporary(info dst_type, info src_type);

// [meta.reflection.unary.prop.query], type property queries
consteval size_t alignment_of(info type);
consteval size_t rank(info type);
consteval size_t extent(info type, unsigned i = 0);

// [meta.reflection.rel], type relations
consteval bool is_same(info type1, info type2);
consteval bool is_base_of(info base_type, info derived_type);
consteval bool is_convertible(info src_type, info dst_type);
consteval bool is_nothrow_convertible(info src_type, info dst_type);
consteval bool is_layout_compatible(info type1, info type2);
consteval bool is_pointer_interconvertible_base_of(info base_type, info
    derived_type);

consteval bool is_invocable(info type, span<const info> type_args);
consteval bool is_invocable_r(info result_type, info type, span<const info>
    type_args);

consteval bool is_nothrow_invocable(info type, span<const info> type_args);
consteval bool is_nothrow_invocable_r(info result_type, info type, span<const info>
    type_args);

// [meta.reflection.trans.cv], const-volatile modifications
consteval info remove_const(info type);
consteval info remove_volatile(info type);
consteval info remove_cv(info type);
consteval info add_const(info type);
consteval info add_volatile(info type);
consteval info add_cv(info type);

// [meta.reflection.trans.ref], reference modifications
consteval info remove_reference(info type);
consteval info add_lvalue_reference(info type);
consteval info add_rvalue_reference(info type);

// [meta.reflection.trans.sign], sign modifications
consteval info make_signed(info type);
consteval info make_unsigned(info type);

// [meta.reflection.trans.arr], array modifications
consteval info remove_extent(info type);
consteval info remove_all_extents(info type);

// [meta.reflection.trans.ptr], pointer modifications
consteval info remove_pointer(info type);
consteval info add_pointer(info type);

// [meta.reflection.trans.other], other transformations
consteval info remove_cvref(info type);
consteval info decay(info type);
consteval info common_type(span<const info> type_args);
consteval info common_reference(span<const info> type_args);
consteval info underlying_type(info type);

```

```

consteval info invoke_result(info type, span<const info> type_args);
consteval info unwrap_reference(info type);
consteval info unwrap_ref_decay(info type);
}

```

§ 5.2.3 [meta.reflection.names] Reflection names and locations

```

consteval string_view name_of(info r);
consteval string_view qualified_name_of(info r);

```

- 1 *Returns:* If *r* designates a declared entity *X*, then the unqualified and qualified names of *X*, respectively. Otherwise, an empty `string_view`.

```

consteval string_view display_name_of(info r);

```

- 2 *Returns:* An implementation-defined string suitable for identifying the reflected construct.

```

consteval source_location source_location_of(info r);

```

- 3 *Returns:* An implementation-defined `source_location` corresponding to the reflected construct.

§ 5.2.4 [meta.reflection.queries] Reflection queries

```

consteval bool is_public(info r);
consteval bool is_protected(info r);
consteval bool is_private(info r);

```

- 1 *Returns:* `true` if *r* designates a class member or base class that is public, protected, or private, respectively. Otherwise, `false`.

```

consteval bool is_accessible(info r);

```

- 2 *Returns:* `TODO`

```

consteval bool is_virtual(info r);

```

- 3 *Returns:* `true` if *r* designates a either a virtual member function or a virtual base class. Otherwise, `false`.

```

consteval bool is_pure_virtual(info r);
consteval bool is_override(info r);

```

- 4 *Returns:* `true` if *r* designates a member function that is pure virtual or overrides another member function, respectively. Otherwise, `false`.

```

consteval bool is_deleted(info r);

```

- 5 *Returns:* `true` if *r* designates a function or member function that is defined as deleted. Otherwise, `false`.

```

consteval bool is_defaulted(info r);

```

- 6 *Returns:* `true` if *r* designates a member function that is defined as defaulted. Otherwise, `false`.

```

consteval bool is_explicit(info r);

```

- 7 *Returns:* `true` if *r* designates a member function that is declared explicit. Otherwise, `false`.

```

consteval bool is_bit_field(info r);

```

- 8 *Returns:* `true` if *r* designates a bit-field. Otherwise, `false`.

```
constexpr bool has_static_storage_duration(info r);
```

9 *Returns:* true if *r* designates an object that has static storage duration. Otherwise, false.

```
constexpr bool has_internal_linkage(info r);  
constexpr bool has_external_linkage(info r);  
constexpr bool has_linkage(info r);
```

10 *Returns:* true if *r* designates an entity that has internal linkage, external linkage, or any linkage, respectively ([basic.link]). Otherwise, false.

```
constexpr bool is_namespace(info r);
```

11 *Returns:* true if *r* designates a namespace or namespace alias. Otherwise, false.

```
constexpr bool is_function(info r);
```

12 *Returns:* true if *r* designates a function or member function. Otherwise, false.

```
constexpr bool is_variable(info r);
```

13 *Returns:* true if *r* designates a variable. Otherwise, false.

```
constexpr bool is_type(info r);
```

14 *Returns:* true if *r* designates a type or a type alias. Otherwise, false.

```
constexpr bool is_alias(info r);
```

15 *Returns:* true if *r* designates a type alias, alias template, or namespace alias. Otherwise, false.

```
constexpr bool is_incomplete_type(info r);
```

16 *Returns:* true if `delias(r)` designates an incomplete type. Otherwise, false.

```
constexpr bool is_template(info r);
```

17 *Returns:* true if *r* designates a function template, class template, variable template, or alias template. Otherwise, false.

18 [*Note 1:* A template specialization is not a template. `is_template(^(std::vector))` is true but `is_template(^(std::vector<int>))` is false. — *end note*]

```
constexpr bool is_function_template(info r);  
constexpr bool is_variable_template(info r);  
constexpr bool is_class_template(info r);  
constexpr bool is_alias_template(info r);  
constexpr bool is_concept(info r);
```

19 *Returns:* true if *r* designates a function template, class template, variable template, alias template, or concept, respectively. Otherwise, false.

```
constexpr bool has_template_arguments(info r);
```

20 *Returns:* true if *r* designates an instantiation of a function template, variable template, class template, or an alias template. Otherwise, false.

```
constexpr auto is_class_member(info entity) -> bool;  
constexpr auto is_namespace_member(info entity) -> bool;  
constexpr bool is_nonstatic_data_member(info r);
```

```

constexpr bool is_static_member(info r);
constexpr bool is_base(info r);
constexpr bool is_constructor(info r);
constexpr bool is_destructor(info r);
constexpr bool is_special_member(info r);

```

21 *Returns:* `true` if `r` designates a class member, namespace member, non-static data member, static member, base class member, constructor, destructor, or special member, respectively. Otherwise, `false`.

```
constexpr info type_of(info r);
```

22 *Mandates:* `r` designates a typed entity.

23 *Returns:* A reflection of the type of that entity.

```
constexpr info parent_of(info r);
```

24 *Mandates:* `r` designates a member of a class or a namespace.

25 *Returns:* A reflection of the that entity's immediately enclosing class or namespace.

```
constexpr info dealias(info r);
```

26 *Returns:* If `r` designates a type alias or a namespace alias, a reflection designating the underlying entity. Otherwise, `r`.

27 *[Example*

```

using X = int;
using Y = X;
static_assert(dealias(^int) == ^int);
static_assert(dealias(^X) == ^int);
static_assert(dealias(^Y) == ^int);

```

-end example]

```

constexpr info template_of(info r);
constexpr vector<info> template_arguments_of(info r);

```

28 *Mandates:* `has_template_arguments(r)` is `true`.

29 *Returns:* A reflection of the template of `r`, and the reflections of the template arguments of the specialization designated by `r`, respectively.

30 *[Example:*

```

template <class T, class U=T> struct Pair { };
template <class T> using PairPtr = Pair<T*>;

static_assert(template_of(^Pair<int>) == ^Pair);
static_assert(template_arguments_of(^Pair<int>).size() == 2);

static_assert(template_of(^PairPtr<int>) == ^PairPtr);
static_assert(template_arguments_of(^PairPtr<int>).size() == 1);

```

-end example]

§ 5.2.5 [meta.reflection.member.queries], Reflection member queries

```
template<class... Fs>
    consteval vector<info> members_of(info r, Fs... filters);
```

1 *Mandates:* `r` is a reflection designating either a class type or a namespace and `(std::predicate<Fs, info> && ...)` is `true`.

2 *Returns:* A vector containing the reflections of all the direct members `m` of the entity designated by `r` such that `(filters(m) && ...)` is `true`. Data members are returned in the order in which they are declared, but the order of member functions and member types is unspecified. [*Note 1:* Base classes are not members. — *end note*]

```
template<class... Fs>
    consteval vector<info> bases_of(info type, Fs... filters);
```

3 *Mandates:* `type` designates a type and `(std::predicate<Fs, info> && ...)` is `true`.

4 *Returns:* Let `C` be the type designated by `type`. A vector containing the reflections of all the direct base classes, if any, of `C` such that `(filters(class_type) && ...)` is `true`. The base classes are returned in the order in which they appear the *base-specifier-list* of `C`.

```
consteval vector<info> static_data_members_of(info type);
```

5 *Mandates:* `type` designates a type.

6 *Effects:* Equivalent to: `return members_of(type, is_variable);`

```
consteval vector<info> nonstatic_data_members_of(info type);
```

7 *Mandates:* `type` designates a type.

8 *Effects:* Equivalent to: `return members_of(type, is_nonstatic_data_member);`

```
consteval vector<info> subobjects_of(info type);
```

9 *Mandates:* `type` designates a type.

10 *Returns:* A vector containing all the reflections in `bases_of(type)` followed by all the reflections in `nonstatic_data_members_of(type)`.

```
consteval vector<info> enumerators_of(info enum_type);
```

11 *Mandates:* `enum_type` designates an enumeration.

12 *Returns:* A vector containing the reflections of each enumerator of the enumeration designated by `enum_type`, in the order in which they are declared.

§ 5.2.6 [meta.reflection.unary] Unary type traits

1 Subclause [meta.reflection.unary] contains consteval functions that may be used to query the properties of a type at compile time.

2 For each function taking an argument of type `meta::info` whose name contains `type`, that argument shall be a reflection of a type or type alias. For each function taking an argument of type `span<const meta::info>` named `type_args`, each `meta::info` in that span shall be a reflection of a type or a type alias.

§ 5.2.6.1 [meta.reflection.unary.cat] Primary type categories

- 1 For any type T , for each function `std::meta::TRAIT` defined in this clause, `std::meta::TRAIT(^T)` equals the value of the corresponding unary type trait `std::TRAIT_v<T>` as specified in 21.3.5.2 [[meta.unary.cat](#)].

```
constexpr bool is_void(info type);
constexpr bool is_null_pointer(info type);
constexpr bool is_integral(info type);
constexpr bool is_floating_point(info type);
constexpr bool is_array(info type);
constexpr bool is_pointer(info type);
constexpr bool is_lvalue_reference(info type);
constexpr bool is_rvalue_reference(info type);
constexpr bool is_member_object_pointer(info type);
constexpr bool is_member_function_pointer(info type);
constexpr bool is_enum(info type);
constexpr bool is_union(info type);
constexpr bool is_class(info type);
constexpr bool is_function(info type);
```

- 2 [Example

```
// an example implementation
namespace std::meta {
    constexpr bool is_void(info type) {
        return value_of<bool>(substitute(^is_void_v, {type}));
    }
}

-end example]
```

§ 5.2.6.2 [meta.reflection.unary.comp] Composite type categories

- 1 For any type T , for each function `std::meta::TRAIT` defined in this clause, `std::meta::TRAIT(^T)` equals the value of the corresponding unary type trait `std::TRAIT_v<T>` as specified in 21.3.5.3 [[meta.unary.comp](#)].

```
constexpr bool is_reference(info type);
constexpr bool is_arithmetic(info type);
constexpr bool is_fundamental(info type);
constexpr bool is_object(info type);
constexpr bool is_scalar(info type);
constexpr bool is_compound(info type);
constexpr bool is_member_pointer(info type);
```

§ 5.2.6.3 [meta.reflection.unary.prop] Type properties

- 1 For any type T , for each function `std::meta::UNARY-TRAIT` defined in this clause with signature `bool(std::meta::info)`, `std::meta::UNARY-TRAIT(^T)` equals the value of the corresponding type property `std::UNARY-TRAIT_v<T>` as specified in 21.3.5.4 [[meta.unary.prop](#)].
- 2 For any types T and U , for each function `std::meta::BINARY-TRAIT` defined in this clause with signature `bool(std::meta::info, std::meta::info)`, `std::meta::BINARY-TRAIT(^T, ^U)` equals the value of the corresponding type property `std::BINARY-TRAIT_v<T, U>` as specified in 21.3.5.4 [[meta.unary.prop](#)].
- 3 For any type T and pack of types $U \dots$, for each function `std::meta::VARIADIC-TRAIT` defined in this clause with signature `bool(std::meta::info, std::span<const std::meta::info>)`,

`std::meta::VARIADIC-TRAIT(^T, {^U...})` equals the value of the corresponding type property `std::VARIADIC-TRAIT_v<T, U...>` as specified in 21.3.5.4 [\[meta.unary.prop\]](#).

```
constexpr bool is_const(info type);
constexpr bool is_volatile(info type);
constexpr bool is_trivial(info type);
constexpr bool is_trivially_copyable(info type);
constexpr bool is_standard_layout(info type);
constexpr bool is_empty(info type);
constexpr bool is_polymorphic(info type);
constexpr bool is_abstract(info type);
constexpr bool is_final(info type);
constexpr bool is_aggregate(info type);
constexpr bool is_signed(info type);
constexpr bool is_unsigned(info type);
constexpr bool is_bounded_array(info type);
constexpr bool is_unbounded_array(info type);
constexpr bool is_scoped_enum(info type);

constexpr bool is_constructible(info type, span<info const> type_args);
constexpr bool is_default_constructible(info type);
constexpr bool is_copy_constructible(info type);
constexpr bool is_move_constructible(info type);

constexpr bool is_assignable(info dst_type, info src_type);
constexpr bool is_copy_assignable(info type);
constexpr bool is_move_assignable(info type);

constexpr bool is_swappable_with(info dst_type, info src_type);
constexpr bool is_swappable(info type);

constexpr bool is_destructible(info type);

constexpr bool is_trivially_constructible(info type, span<info const> type_args);
constexpr bool is_trivially_default_constructible(info type);
constexpr bool is_trivially_copy_constructible(info type);
constexpr bool is_trivially_move_constructible(info type);

constexpr bool is_trivially_assignable(info dst_type, info src_type);
constexpr bool is_trivially_copy_assignable(info type);
constexpr bool is_trivially_move_assignable(info type);
constexpr bool is_trivially_destructible(info type);

constexpr bool is_nothrow_constructible(info type, span<info const> type_args);
constexpr bool is_nothrow_default_constructible(info type);
constexpr bool is_nothrow_copy_constructible(info type);
constexpr bool is_nothrow_move_constructible(info type);

constexpr bool is_nothrow_assignable(info dst_type, info src_type);
constexpr bool is_nothrow_copy_assignable(info type);
constexpr bool is_nothrow_move_assignable(info type);

constexpr bool is_nothrow_swappable_with(info dst_type, info src_type);
constexpr bool is_nothrow_swappable(info type);

constexpr bool is_nothrow_destructible(info type);

constexpr bool is_implicit_lifetime(info type);

constexpr bool has_virtual_destructor(info type);
```

```

consteval bool has_unique_object_representations(info type);

consteval bool reference_constructs_from_temporary(info dst_type, info src_type);
consteval bool reference_converts_from_temporary(info dst_type, info src_type);

```

§ 5.2.6.4 [meta.reflection.unary.prop.query] Type property queries

- 1 For any type T , for each function `std::meta::PROP` defined in this clause with signature `size_t(std::meta::info)`, `std::meta::PROP(^T)` equals the value of the corresponding type property `std::PROP_v<T>` as specified in 21.3.6 [meta.unary.prop.query].
- 2 For any type T and unsigned integer value I , `std::meta::extent(^T, I)` equals `std::extent_v<T, I>` ([meta.unary.prop.query]).

```

consteval size_t alignment_of(info type);
consteval size_t rank(info type);
consteval size_t extent(info type, unsigned i = 0);

```

§ 5.2.7 [meta.reflection.rel], Type relations

- 1 The consteval functions specified in this clause may be used to query relationships between types at compile time.
- 2 For any types T and U , for each function `std::meta::REL` defined in this clause with signature `bool(std::meta::info, std::meta::info)`, `std::meta::REL(^T, ^U)` equals the value of the corresponding type relation `std::REL_v<T, U>` as specified in 21.3.7 [meta.rel].
- 3 For any type T and pack of types $U...$, for each function `std::meta::VARIADIC-REL` defined in this clause with signature `bool(std::meta::info, std::span<const std::meta::info>)`, `std::meta::VARIADIC-REL(^T, {^U...})` equals the value of the corresponding type relation `std::VARIADIC-REL_v<T, U...>` as specified in 21.3.7 [meta.rel].
- 4 For any types T and R and pack of types $U...$, for each function `std::meta::VARIADIC-REL-R` defined in this clause with signature `bool(std::meta::info, std::meta::info, std::span<const std::meta::info>)`, `std::meta::VARIADIC-REL-R(^R, ^T, {^U...})` equals the value of the corresponding type relation `std::VARIADIC-REL-R_v<R, T, U...>` as specified in 21.3.7 [meta.rel].

```

consteval bool is_same(info type1, info type2);
consteval bool is_base_of(info base_type, info derived_type);
consteval bool is_convertible(info src_type, info dst_type);
consteval bool is_nothrow_convertible(info src_type, info dst_type);
consteval bool is_layout_compatible(info type1, info type2);
consteval bool is_pointer_interconvertible_base_of(info base_type, info derived_type);

consteval bool is_invocable(info type, span<const info> type_args);
consteval bool is_invocable_r(info result_type, info type, span<const info>
    type_args);

consteval bool is_nothrow_invocable(info type, span<const info> type_args);
consteval bool is_nothrow_invocable_r(info result_type, info type, span<const info>
    type_args);

```

- 5 [Note 1: If t is a reflection of the type `int` and u is a reflection of an alias to the type `int`, then `t == u` is `false` but `is_same(t, u)` is `true`. `t == dealias(u)` is also `true`. — end note].

§ 5.2.8 [meta.reflection.trans], Transformations between types

- 1 Subclause [meta.reflection.trans] contains consteval functions that may be used to transform one type to another following some predefined rule.

§ 5.2.8.1 [meta.reflection.trans.cv], Const-volatile modifications

- 1 For any type T , for each function `std::meta::MOD` defined in this clause, `std::meta::MOD(^T)` returns the reflection of the corresponding type `std::MOD_t<T>` as specified in 21.3.8.2 [meta.trans.cv].

```
consteval info remove_const(info type);
consteval info remove_volatile(info type);
consteval info remove_cv(info type);
consteval info add_const(info type);
consteval info add_volatile(info type);
consteval info add_cv(info type);
```

§ 5.2.8.2 [meta.reflection.trans.ref], Reference modifications

- 1 For any type T , for each function `std::meta::MOD` defined in this clause, `std::meta::MOD(^T)` returns the reflection of the corresponding type `std::MOD_t<T>` as specified in 21.3.8.3 [meta.trans.ref].

```
consteval info remove_reference(info type);
consteval info add_lvalue_reference(info type);
consteval info add_rvalue_reference(info type);
```

§ 5.2.8.3 [meta.reflection.trans.sign], Sign modifications

- 1 For any type T , for each function `std::meta::MOD` defined in this clause, `std::meta::MOD(^T)` returns the reflection of the corresponding type `std::MOD_t<T>` as specified in 21.3.8.4 [meta.trans.sign].

```
consteval info make_signed(info type);
consteval info make_unsigned(info type);
```

§ 5.2.8.4 [meta.reflection.trans.arr], Array modifications

- 1 For any type T , for each function `std::meta::MOD` defined in this clause, `std::meta::MOD(^T)` returns the reflection of the corresponding type `std::MOD_t<T>` as specified in 21.3.8.5 [meta.trans.arr].

```
consteval info remove_extent(info type);
consteval info remove_all_extents(info type);
```

§ 5.2.8.5 [meta.reflection.trans.ptr], Pointer modifications

- 1 For any type T , for each function `std::meta::MOD` defined in this clause, `std::meta::MOD(^T)` returns the reflection of the corresponding type `std::MOD_t<T>` as specified in 21.3.8.6 [meta.trans.ptr].

```
consteval info remove_pointer(info type);
consteval info add_pointer(info type);
```

§ 5.2.8.6 [meta.reflection.trans.other], Other transformations

[Editor's note: There are four transformations that are deliberately omitted here. `type_identity` and `enable_if` are not useful, `conditional(cond, t, f)` would just be a long way of writing `cond ? t : f`, and `basic_common_reference` is a class template intended to be specialized and not directly invoked.]

- 1 For any type T , for each function `std::meta::MOD` defined in this clause with signature `std::meta::info(std::meta::info)`, `std::meta::MOD(^T)` returns the reflection of the corresponding

type `std::MOD_t<T>` as specified in 21.3.8.7 [meta.trans.other].

- 2 For any pack of types $T \dots$, for each function `std::meta::VARIADIC-MOD` defined in this clause with signature `std::meta::info(std::span<const std::meta::info>)`, `std::meta::VARIADIC-MOD({^T...})` returns the reflection of the corresponding type `std::VARIADIC-MOD_t<T...>` as specified in 21.3.8.7 [meta.trans.other].
- 3 For any type T and pack of types $U \dots$, `std::meta::invoke_result(^T, {^U...})` returns the reflection of the corresponding type `std::invoke_result_t<T, U...>` (21.3.8.7 [meta.trans.other]).

```
constexpr info remove_cvref(info type);
constexpr info decay(info type);
constexpr info common_type(span<const info> type_args);
constexpr info common_reference(span<const info> type_args);
constexpr info underlying_type(info type);
constexpr info invoke_result(info type, span<const info> type_args);
constexpr info unwrap_reference(info type);
constexpr info unwrap_ref_decay(info type);
```

- 4 [Example:

```
// example implementation
constexpr info unwrap_reference(info type) {
    if (has_template_arguments(type) && template_of(type) == ^reference_wrapper) {
        return add_lvalue_reference(template_arguments_of(type)[0]);
    } else {
        return type;
    }
}

-end example]
```

§ 6 References

[N3980] H. Hinnant, V. Falco, J. Byteway. 2014-05-24. Types don't know #.

<https://wg21.link/n3980>

[P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More constexpr containers.

<https://wg21.link/p0784r7>

[P1061R5] Barry Revzin, Jonathan Wakely. 2023-05-18. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r5>

[P1240R0] Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2018-10-08. Scalable Reflection in C++.

<https://wg21.link/p1240r0>

[P1240R2] Daveed Vandevoorde, Wyatt Childers, Andrew Sutton, Faisal Vali. 2022-01-14. Scalable Reflection.

<https://wg21.link/p1240r2>

[P1306R1] Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2019-01-21. Expansion statements.

<https://wg21.link/p1306r1>

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using `propconst`.

<https://wg21.link/p1974r0>

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.

<https://wg21.link/p2237r0>

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.

<https://wg21.link/p2670r1>

[P2758R1] Barry Revzin. 2023-12-09. Emitting messages at compile time.

<https://wg21.link/p2758r1>

[P2996R0] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2023-10-15. Reflection for C++26.

<https://wg21.link/p2996r0>

[P2996R1] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde. 2023-12-18. Reflection for C++26.

<https://wg21.link/p2996r1>